

BW-KEM

Algorithm Specifications and Supporting Documentation

(version 1.0)

Yunlei Zhao, Songlin Li, Hengchuan Zou, Chaofeng Guan,
Yanbin Pan, Weizhi Ao, Zhenhao Zhang, Zhenyu Song, Han Liu,
Shuaihao Li, Hanyu Wei, Xinsheng Zhang, and Wenbo Guo

June 29, 2026

Table of Contents

1	Introduction.....	3
1.1	Claimed Characteristics of BW-KEM	3
1.2	Document Organization	4
2	Preliminaries	5
2.1	Notations	5
2.2	Cryptographic Primitives	6
2.3	Hardness assumption	7
3	The Lattice Code	7
3.1	Barnes-Wall Lattices	8
3.2	Scalable Lattice Coding Based on Barnes-Wall Lattices	9
4	Construction and Analysis	14
4.1	The IND-CPA-secure PKE	14
4.2	The IND-CCA-secure KEM	19
5	Analyzing the DFR without Independence Assumption	21
5.1	The Joint Distribution of Coefficients in the Noise Polynomial ..	21
5.2	Asymptotic Approximation of the Noise Distribution for DFR Estimation	24
6	Concrete Security and Parameter Selection	26
6.1	Core-SVP Hardness	26
6.2	Parameter Sets	27
7	Implementation and Benchmark	28
7.1	Implementation Overview	28
7.2	Performance and Benchmark Results	29
7.3	ARM Cortex-M4 Implementation and Benchmarking	30
A	Properties of the Matrix $\mathbf{W}_n$	35

1 Introduction

BW-KEM is a lattice-based key encapsulation mechanism (KEM) designed for post-quantum cryptographic standardization, which has first been described in [15]. The security of BW-KEM is based on the hardness of solving the learning-with-errors problem in module lattices (MLWE problem [14]). The core of BW-KEM consists of three main components: a scalable nested lattice coding framework based on Barnes-Wall lattices that enables efficient and reliable message recovery, an IND-CPA-secure public-key encryption scheme BW-PKE, and an IND-CCA-secure KEM scheme BW-KEM constructed by applying a variant of the Fujisaki-Okamoto (FO) transformation to BW-PKE.

1.1 Claimed Characteristics of BW-KEM

BW-KEM exhibits the following main characteristics.

Innovation in Error Correction and DFR Analysis. BW-KEM introduces a novel unified framework that integrates structured, low-dimensional Barnes-Wall lattice codes into high-dimensional module-lattice constructions. Unlike conventional approaches that rely on direct modulation or concatenation with classical error-correcting codes, BW-KEM adopts a scalable nested lattice coding scheme as the core error correction mechanism. This design not only enhances error tolerance but also enables a rigorous analysis of the decryption failure rate (DFR) without relying on independence assumptions on noise coefficients. A preliminary version [15] of BW-KEM appears in PKC 2026.

Simplicity in Algorithmic Design. The overall construction of BW-KEM remains conceptually simple and clean. The scheme preserves the core structure of MLWE-based KEMs while introducing only a lightweight modification in the error-correction component. The encoding and decoding procedures are explicitly defined, algebraically structured, and suitable for constant-time implementation. Unlike traditional concatenated ECC schemes that introduce high structural complexities and side-channel vulnerabilities, BW-KEM maintains profound algorithmic elegance. In particular, by intelligently adopting power-of-two scaling factors, the decoding procedure avoids all fixed-point or rational arithmetic. All internal operations are executed using pure integer arithmetic and lightweight bitwise logic, naturally facilitating straightforward, constant-time implementations. This simplicity makes BW-KEM practical for both software and hardware deployments.

Flexibility. BW-KEM exhibits a high degree of flexibility due to its scalable lattice coding framework. The parameters of the nested lattice code, such as dimension and scaling factors, can be adjusted to accommodate different security levels and application requirements. This flexibility enables fine-grained

trade-offs among concrete security margins, ciphertext compactness, error resilience and computational efficiency. Moreover, the modular design allows the lattice coding component to be adapted or extended independently, facilitating integration into a wide range of cryptographic settings.

Compatibility with Existing Standards. BW-KEM is fully compatible with existing MLWE-based cryptographic frameworks, particularly ML-KEM. The proposed scheme can be viewed as a drop-in enhancement that replaces the original error-correction mechanism without altering the overall structure of key generation, encapsulation, and decapsulation. Crucially, BW-KEM utilizes the identical modulus q and polynomial dimensions as ML-KEM, enabling the direct reuse of the vast majority of existing software modules and hardware acceleration circuits (such as number-theoretic transform units and arithmetic pipelines). This shared algebraic foundation significantly promotes implementation simplicity, minimizes engineering overhead, and seamlessly facilitates the mutual transformation and smooth migration between international and domestic cryptographic algorithm standards. Such compatibility ensures that BW-KEM can be effortlessly integrated into legacy infrastructures and existing standards, preserving interoperability while delivering superior performance and robustness.

Scalability. A key advantage of BW-KEM lies in its inherent scalability. The Barnes–Wall lattice structure enables recursive construction and efficient implementation across multiple dimensions, allowing the scheme to scale naturally with increasing security requirements. The scalable nested lattice coding framework supports efficient parameter tuning and maintains performance consistency across different configurations. This property makes BW-KEM well-suited for long-term deployment in diverse application scenarios.

1.2 Document Organization

The remainder of this document is organized as follows. Section 2 reviews the necessary notations, cryptographic primitives, and the module learning with errors (MLWE) hardness assumption. Section 3 details the design of the scalable nested lattice code based on Barnes–Wall lattices, providing its constant-time encoding and decoding algorithms along with formal proofs of correctness. Section 4 presents the formal specification of BW-KEM, including the IND-CPA-secure public-key encryption scheme and its transformation into an IND-CCA-secure key encapsulation mechanism. Section 5 analyzes the decryption failure rate (DFR) without assuming independence among noise coefficients, and derives an asymptotic approximation for efficient DFR estimation. Section 6 discusses concrete security evaluations under the core-SVP hardness model and presents the recommended parameter sets for different security levels. Section 7 describes the implementation aspects and reports performance benchmarks.

2 Preliminaries

2.1 Notations

Let $\mathbb{Z}$, $\mathbb{R}$ and $\mathbb{C}$ be the sets of rational integers, real numbers and complex numbers, respectively. Let N and q be positive integers. $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z} \cong \{0, 1, \dots, q-1\}$ denotes the residue class modulo q . Let $\mathcal{R}$ and $\mathcal{R}_q$ denote the rings $\mathbb{Z}[x]/(x^N + 1)$ and $\mathbb{Z}_q[x]/(x^N + 1)$, respectively, where $N = 2^{k_0}$ such that $x^N + 1$ is the $2N$ -th cyclotomic polynomial. Regular font letters denote elements in $\mathcal{R}$ or $\mathcal{R}_q$, bold lower-case letters represent vectors and bold upper-case letters are matrices. For a vector $\mathbf{v}$ (resp., matrix $\mathbf{A}$), we denote $\mathbf{v}^T$ (resp., matrix $\mathbf{A}^T$) by its transpose. For a polynomial a , we denote by a_i the i -th coefficient associated with the power x^i . For any $x \in \mathbb{R}$, we denote by $\lfloor x \rfloor$ the operation of rounding x to the nearest integer, with ties being rounded up.

Modular reductions. In this work, we extend the definition of modular reduction from $\mathbb{Z}$ to $\mathbb{R}$. For a positive integer q , we define the modular reduction of r under the signed reduction convention as $r' = r \bmod^{\pm} q$, where r' is the unique representative of r in the interval $[-\frac{q}{2}, \frac{q}{2})$. Similarly, under the standard (non-negative) modular reduction, we define $r' = r \bmod q$, where r' is the unique representative of r in the interval $[0, q)$.

Sizes of elements. For $\omega \in \mathbb{R}$, its l_{∞} norm is denoted as $\|\omega\|_{q,\infty} = |\omega \bmod^{\pm} q|$. For an N -dimensional vector $\mathbf{v} = (v_0, \dots, v_{N-1}) \in \mathbb{R}^N$, we define its l_2 norm as $\|\mathbf{v}\|_{q,2} = \sqrt{\|v_0\|_{q,\infty}^2 + \dots + \|v_{N-1}\|_{q,\infty}^2}$. If $\omega = \omega_0 + \omega_1 x + \dots + \omega_{N-1} x^{N-1} \in \mathcal{R}$ or $\mathcal{R}_q$, then its l_2 norm is defined as $\|\omega\|_{q,2} = \sqrt{\|\omega_0\|_{q,\infty}^2 + \dots + \|\omega_{N-1}\|_{q,\infty}^2}$. Similarly, for $\mathbf{w} = (\omega_1, \dots, \omega_l) \in \mathcal{R}^l$ or $\mathcal{R}_q^l$, we define its l_2 norm as $\|\mathbf{w}\|_{q,2} = \sqrt{\|\omega_1\|_{q,2}^2 + \dots + \|\omega_l\|_{q,2}^2}$.

Distributions. For a set S , the notation $x \stackrel{\$}{\leftarrow} S$ denotes that x is sampled uniformly at random from S . If S is a probability distribution, then $x \leftarrow S$ indicates that x is drawn according to the distribution S .

The centered binomial distribution $\mathcal{B}_{\eta}$, parameterized by a positive integer η , is defined as follows: sample $(a_1, \dots, a_{\eta}, b_1, \dots, b_{\eta}) \stackrel{\$}{\leftarrow} \{0, 1\}^{2\eta}$ and output $\sum_{i=1}^{\eta} (a_i - b_i)$. For an element $f \in \mathcal{R}$, the notation $f \leftarrow \beta_{\eta}$ indicates that f is generated by independently sampling each coefficient of f from the distribution $\mathcal{B}_{\eta}$.

Compression and Decompression. We define the compression function $y = \text{Compress}_{q,2^d}(x)$, which maps an element $x \in \mathbb{Z}_q$ to an integer y in the range $\{0, 1, \dots, 2^d - 1\}$, where the positive integer $d < \lceil \log_2(q) \rceil$. We further define the corresponding decompression function $x' = \text{Decompress}_{2^d,q}(y)$ that outputs an element x' close to the original x . Specifically, these functions are defined as:

$$\begin{aligned} \text{Compress}_{q,2^d}(x) &= \left\lfloor \frac{2^d}{q} \cdot x \right\rfloor \bmod 2^d, \\ \text{Decompress}_{2^d,q}(y) &= \left\lfloor \frac{q}{2^d} \cdot y \right\rfloor. \end{aligned}$$

The error distribution of $x' - x \bmod \pm q$ is nearly uniform over the integers with magnitude at most $\lfloor \frac{q}{2^{d+1}} \rfloor$. For a polynomial $\omega \in \mathcal{R}_q$ or a vector $\mathbf{w} \in \mathcal{R}_q^k$, these operations are applied coefficient-wise.

The primary purpose of defining the $\text{Compress}_{q,2^d}$ and $\text{Decompress}_{2^d,q}$ functions is to enable the removal of a certain number of low-order bits from the ciphertext. These discarded bits contribute minimally to the correctness probability of decryption while significantly reducing the overall parameter sizes, thereby improving efficiency in terms of storage and communication overhead.

2.2 Cryptographic Primitives

Public-Key Encryption A public-key encryption (PKE) scheme is a tuple of three algorithms $\text{PKE} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ along with a message space $\mathcal{M}$.

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$: The key generation algorithm KeyGen outputs a key pair (pk, sk) , where pk also defines a finite randomness space $\mathcal{R} = \mathcal{R}(\text{pk})$ as well as a ciphertext space $\mathcal{C}$.
- $\text{ct} \leftarrow \text{Enc}(\text{pk}, m)$: The encryption algorithm Enc takes as inputs the public key pk and a message $m \in \mathcal{M}$ and outputs a ciphertext ct . If necessary, we make the used randomness of encryption explicit by writing $\text{ct} := \text{Enc}(\text{pk}, m; r)$, where $r \in \mathcal{R}$.
- $m' \leftarrow \text{Dec}(\text{sk}, \text{ct})$: The decryption algorithm Dec , on input sk and a ciphertext ct , deterministically outputs either a message $m' \in \mathcal{M}$ or a special symbol $\perp \notin \mathcal{M}$ to indicate that ct is not a valid ciphertext.

A PKE scheme is $(1-\delta)$ -correct if $\mathbb{E}[\max_{m \in \mathcal{M}} \Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m)) = m]] \geq 1 - \delta$, where the expectation is taken over $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$ and the probability is taken over the random coins of Enc . The advantage of an adversary $\mathcal{A}$ against indistinguishability under chosen-plaintext attacks (IND-CPA) for PKE is defined as

$$\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) := \left| \Pr \left[\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}} \Rightarrow 1 \right] - \frac{1}{2} \right|$$

where game $\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}$ is defined in Figure 1.

Game $\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}$	Game $\text{IND-CCA}_{\text{KEM}}^{\mathcal{A}}$
1 : $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$	1 : $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$
2 : $(m_0, m_1) \leftarrow \mathcal{A}(\text{pk})$	2 : $b \xleftarrow{\$} \{0, 1\}$
3 : $b \xleftarrow{\$} \{0, 1\}$	3 : $(\text{ct}^*, K_0^*) \leftarrow \text{Encaps}(\text{pk})$
4 : $\text{ct}^* \leftarrow \text{Enc}(\text{pk}, m_b)$	4 : $K_1^* \xleftarrow{\$} \mathcal{K}$
5 : $b' \leftarrow \mathcal{A}(\text{ct}^*)$	5 : $b' \leftarrow \mathcal{A}^{\text{Decaps}}(\text{pk}, \text{ct}^*, K_b^*)$
6 : return $\llbracket b' = b \rrbracket$	6 : return $\llbracket b' = b \rrbracket$

Fig. 1. IND-CPA game for PKE and IND-CCA game for KEM.

Key Encapsulation Mechanism A key encapsulation mechanism (KEM) scheme consists of three algorithms $\text{KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$ along with a key space $\mathcal{K}$.

- $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$: The key generation algorithm KeyGen outputs a key pair (pk, sk) , where pk also defines a finite randomness space $\mathcal{R} = \mathcal{R}(\text{pk})$ as well as a ciphertext space $\mathcal{C}$.
- $(\text{ct}, K) \leftarrow \text{Encaps}(\text{pk})$: The encryption algorithm Encaps takes a public key pk and produce a key encapsulation ciphertext ct and a key K .
- $K' \leftarrow \text{Decaps}(\text{sk}, \text{ct})$: The decryption algorithm Decaps , on input sk and a ciphertext ct , deterministically outputs a key $K' \in \mathcal{K}$. If the KEM allows explicit rejection, the output is either a key K' or a special symbol $\perp$ to indicate a rejection.

A KEM scheme is $(1 - \delta)$ -correct if $\Pr[\text{Decaps}(\text{sk}, \text{ct}) = K : (\text{ct}, K) \leftarrow \text{Encaps}(\text{pk})] \geq 1 - \delta$, where the probability is taken over $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}()$ and the random coins of Encaps . The advantage of an adversary $\mathcal{A}$ against indistinguishability under chosen-ciphertext attacks (IND-CCA) for KEM is defined as

$$\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}) := \left| \Pr \left[\text{IND-CCA}_{\text{KEM}}^{\mathcal{A}} \Rightarrow 1 \right] - \frac{1}{2} \right|,$$

where game $\text{IND-CCA}_{\text{KEM}}^{\mathcal{A}}$ is defined in Figure 1 and the oracle $\mathcal{O}_{\text{Decaps}}(\text{sk}, \cdot \neq \text{ct}^*)$ returns $\text{Decaps}(\text{sk}, \text{ct})$ for all $\text{ct} \neq \text{ct}^*$.

2.3 Hardness assumption

Module-LWE. Let l, m be positive integer parameters, and let $\mathcal{D}$ be a distribution over the ring $\mathcal{R}_q$. The (decisional) Module Learning With Errors (MLWE) problem is to distinguish between two types of samples: uniformly random pairs $(\mathbf{a}_i, b_i) \xleftarrow{\$} \mathcal{R}_q^l \times \mathcal{R}_q$, and samples where $\mathbf{a}_i \xleftarrow{\$} \mathcal{R}_q^l$ is uniform and $b_i = \mathbf{a}_i^T \mathbf{s} + e_i$, with a secret $\mathbf{s} \leftarrow \mathcal{D}^l$ shared across all samples and independent noise terms $e_i \leftarrow \mathcal{D}$.

The MLWE problem is assumed to be hard, meaning that for any probabilistic polynomial-time adversary $\mathcal{A}$, its distinguishing advantage is negligible, where $\text{Adv}_{m,l,\mathcal{D}}^{\text{MLWE}}(\mathcal{A}) =$

$$\left| \Pr \left[b' = 1 : \begin{array}{l} \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{m \times l}; \\ (\mathbf{s}, \mathbf{e}) \leftarrow \mathcal{D}^l \times \mathcal{D}^m; \\ \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}; \\ b' \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{b}) \end{array} \right] - \Pr \left[b' = 1 : \begin{array}{l} \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{m \times l}; \\ \mathbf{b} \xleftarrow{\$} \mathcal{R}_q^m; \\ b' \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{b}) \end{array} \right] \right|.$$

3 The Lattice Code

Before introducing our MLWE-based KEM scheme, we first establish the necessary theoretical foundations on Barnes-Wall lattices and lattice coding theory.

Subsequently, we propose a simple yet efficient scalable lattice code based on Barnes-Wall lattices, designed to enhance message recovery efficiency while ensuring negligible error probabilities in our scheme.

3.1 Barnes-Wall Lattices

Barnes-Wall lattices are an infinite sequence of full-rank lattices defined for every dimension n that is a power of 2. It is known that Barnes-Wall lattices can be viewed as lattices over the Gaussian integers $\mathbb{G} = \mathbb{Z}[i]$ [18], the construction of which we adopt in this paper.

Definition 1 (Barnes-Wall lattices) *Let $n = 2^k$ ($k \geq 2$) and $\phi = 1 + i$, the Barnes-Wall lattice of dimension n is defined recursively as*

$$\text{BW}_n = \{[\mathbf{u}, \mathbf{u} + \phi\mathbf{v}] \mid \mathbf{u}, \mathbf{v} \in \text{BW}_{\frac{n}{2}}\},$$

with initial condition $\text{BW}_2 = \mathbb{G}$.

Alternatively, the Barnes-Wall lattice can also be defined as lattice over $\mathbb{G}$ generated by the rows of the $(k-1)$ -fold Kronecker product $\mathbf{W}_n$:

$$\text{BW}_n = \{\mathbf{v} \cdot \mathbf{W}_n \mid \mathbf{v} \in \mathbb{G}^{\frac{n}{2}}\}, \mathbf{W}_n = \begin{pmatrix} 1 & 1 \\ 0 & \phi \end{pmatrix}^{\otimes(k-1)} \in \mathbb{C}^{\frac{n}{2} \times \frac{n}{2}}.$$

To obtain the scaled Barnes-Wall lattices is straightforward based on the definition above:

$$\text{BW}_n(\lambda) = \{\lambda\mathbf{v} \cdot \mathbf{W}_n \mid \mathbf{v} \in \mathbb{G}^{\frac{n}{2}}\}, \lambda \in \mathbb{R}^+.$$

Complex numbers and scaled lattices make the definitions more concise, while also enhancing the simplicity and flexibility of subsequent algorithms. The construction can be easily converted to lattices over $\mathbb{Z}$ via the following mapping from $\mathbb{C}^{\frac{n}{2}}$ to $\mathbb{R}^n$:

$$(a_1 + b_1i, a_2 + b_2i, \dots, a_{n/2} + b_{n/2}i) \mapsto (a_1, b_1, a_2, b_2, \dots, a_{n/2}, b_{n/2}).$$

Under this mapping, the norm and distance notations can be naturally extended. For $n = 2, 4, 8, 16$, the Barnes-Wall lattices are equivalent to $\mathbb{Z}^2$, D_4 , E_8 , and A_{16} , respectively [8]. And these are densest packings in respective dimensions.

The results in [11] can be naturally extended to derive the following lemma concerning scaled Barnes-Wall lattices:

Lemma 1 *Let $\text{BW}_n(\lambda)$ be the scaled Barnes-Wall lattice, where $n = 2^k \geq 2$. The following properties hold:*

(1) *The minimum squared distance satisfies*

$$d_{\min}^2(\text{BW}_n(\lambda)) = \frac{n}{2}\lambda^2 = (2\rho(\text{BW}_n(\lambda)))^2,$$

where $\rho(\text{BW}_n(\lambda))$ denotes the packing radius.

(2) *The fundamental volume is given by $\text{Vol}(\text{BW}_n(\lambda)) = \lambda^n \cdot 2^{\frac{n}{4}(k-1)}$.*

(3) *The inclusion relation holds: $2^{\lfloor \frac{k}{2} \rfloor} \lambda \cdot \mathbb{G}^{\frac{n}{2}} \subset \text{BW}_n(\lambda)$.*

3.2 Scalable Lattice Coding Based on Barnes-Wall Lattices

Definition 2 (Nested lattices) *Two lattices Λ_c and Λ_f are nested if $\Lambda_c \subset \Lambda_f$. The lattice Λ_c is referred to as the coarse/shaping lattice, and Λ_f as the fine/coding lattice.*

Definition 3 (Nested lattice code) *Let Λ_c and Λ_f be two lattices with $\Lambda_c \subset \Lambda_f$ and let $\mathcal{P}_c$ be a fundamental region of Λ_c . A nested lattice code $\mathcal{C}$ based on the nested lattice pair is defined as $\mathcal{C} = \Lambda_f \cap \mathcal{P}_c$, which is a subset of Λ_f that forms a complete set of representatives for the quotient group Λ_f/Λ_c .*

When utilizing a nested lattice code $\mathcal{C}$ for message transmission over a noisy channel, the process typically involves the following steps:

- **Encoding:** A lattice labeling process maps each message $\mathbf{m} \in \mathcal{M}$ to a corresponding lattice codeword $\mathbf{w} \in \mathcal{C}$.
- **Lattice Decoding:** Upon receiving the noisy vector $\mathbf{x} = \mathbf{w} + \mathbf{e}$, where $\mathbf{e}$ represents noise, this step employs a certain error-correcting algorithm which treats $\mathbf{x}$ as the input target vector and outputs a lattice vector $\mathbf{w}' \in \mathcal{C}$. Subsequently, a lattice delabeling process maps $\mathbf{w}'$ back to the estimated message $\mathbf{m}'$.

The error-correcting mechanism employed in this work is a Bounded Distance Decoding (BDD) algorithm with radius r . For successful decoding (i.e., ensuring $\mathbf{m}' = \mathbf{m}$), a sufficient condition is that the encoding process is injective and the noise vector lies within the decoding radius r .

In this paper, we consider the coding lattice $\Lambda_f = \text{BW}_n(\lambda)$ and the shaping lattice $\Lambda_c = 2^\tau \lambda \cdot \mathbb{G}^{n/2}$, where $\tau \geq \lfloor \frac{k}{2} \rfloor$. According to Lemma 1, it follows that

$$\Lambda_c \subset 2^{\lfloor \frac{k}{2} \rfloor} \lambda \cdot \mathbb{G}^{n/2} \subset \Lambda_f.$$

Hence, the **scalable nested lattice code**, denoted by $\mathcal{C}_{n,\tau}(\lambda)$, can be defined based on the nested lattice pair $\Lambda_c \subset \Lambda_f$. For the lattice Λ_c , we define $\mathcal{P}_c$ as the fundamental parallelotope corresponding to its standard orthogonal basis. Specifically, it is defined as:

$$\mathcal{P}_c = \{(a_1 + b_1 i, a_2 + b_2 i, \dots, a_{n/2} + b_{n/2} i) \mid 0 \leq a_j, b_j < 2^\tau \lambda, \forall 1 \leq j \leq n/2\}.$$

Applying Lemma 1, we derive the cardinality of $\mathcal{C}_{n,\tau}(\lambda)$ as

$$|\mathcal{C}_{n,\tau}(\lambda)| = \frac{\text{Vol}(\Lambda_c)}{\text{Vol}(\Lambda_f)} = \frac{\lambda^n \cdot 2^{\tau n}}{\lambda^n \cdot 2^{\frac{n}{4}(k-1)}} = 2^{\tau n - \frac{n}{4}(k-1)}. \quad (1)$$

Next, we present the encoding and decoding procedures for the message space $\mathcal{M} = \{0, 1\}^\mu$ and the lattice code $\mathcal{C}_{n,\tau}(\lambda)$. To ensure the injectivity of the encoding map, we impose the condition $|\mathcal{M}| \leq |\mathcal{C}_{n,\tau}(\lambda)|$, which leads to the constraint $\mu \leq \tau n - \frac{n}{4}(k-1)$.

Algorithm 1 $\text{Encode}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{m} \in \{0,1\}^\mu)$

Input: μ -bit string $\mathbf{m} \in \{0,1\}^\mu$, $\mu \leq \tau n - \frac{n}{4}(k-1)$

Output: A lattice code vector $\mathbf{w} \in \mathcal{C}_{n,\tau}(\lambda)$

- 1: Pad $\mathbf{m}$ with 0's to obtain $\bar{\mathbf{m}}$ with length $\tau n - \frac{n}{4}(k-1)$
 - 2: Divide $\bar{\mathbf{m}}$ into $\frac{n}{2}$ blocks: $\bar{\mathbf{m}} = (\mathbf{m}_0, \mathbf{m}_1, \dots, \mathbf{m}_{\frac{n}{2}-1})$, such that $\mathbf{m}_j \in \{0,1\}^{2\tau-w_H(j)}$
 - 3: **for** j from 0 to $\frac{n}{2} - 1$ **do**
 - 4: $v_j := \xi_{2\tau-w_H(j)}(\mathbf{m}_j)$
 - 5: **end for**
 - 6: $\mathbf{v} := (v_0, \dots, v_{\frac{n}{2}-1})$
 - 7: $\mathbf{w} := \lambda \cdot (\mathbf{v} \mathbf{W}_n \bmod 2^\tau)$
 - 8: **return** $\mathbf{w}$
-

Encoding The encoding algorithm first pads the message $\mathbf{m}$ to obtain $\bar{\mathbf{m}}$ with length $\tau n - \frac{n}{4}(k-1)$ by appending zeros. Then, in the second step, $\bar{\mathbf{m}}$ is divided into $\frac{n}{2}$ sub-vectors, and each component $\mathbf{m}_j$ is mapped to a Gaussian integer v_j , where $w_H(j)$ denotes the Hamming weight of j . It is straightforward to verify that the total length of $\mathbf{m}_j$ satisfies

$$\sum_{j=0}^{\frac{n}{2}-1} (2\tau - w_H(j)) = \tau n - \frac{n}{4}(k-1),$$

ensuring that the partition is well-defined. The mapping is defined as $\xi_l : \{0,1\}^l \rightarrow \mathcal{V}_l$, where $\mathcal{V}_l = \{a + bi \in \mathbb{G} \mid 0 \leq a < 2^{\lceil \frac{l}{2} \rceil}, 0 \leq b < 2^{\lfloor \frac{l}{2} \rfloor}\}$. Denote $\omega = (\omega_0, \omega_1, \dots, \omega_{l-1}) \in \{0,1\}^l$. ξ_l encodes the low $\lceil \frac{l}{2} \rceil$ bits as a , while the remaining bits as b , i.e.,

$$\begin{aligned} \xi_l : \{0,1\}^l &\rightarrow \mathcal{V}_l \\ \omega = (\omega_0, \omega_1, \dots, \omega_{l-1}) &\mapsto \sum_{j=0}^{\lceil \frac{l}{2} \rceil - 1} \omega_j \cdot 2^j + i \sum_{j=\lceil \frac{l}{2} \rceil}^{l-1} \omega_j \cdot 2^{j - \lceil \frac{l}{2} \rceil} \end{aligned}$$

Finally, the encoded vector $\mathbf{w}$ is computed as $\mathbf{w} = \lambda \cdot (\mathbf{v} \mathbf{W}_n \bmod 2^\tau)$, resulting in a vector corresponding to $\mathbf{m}$ in the scalable nested lattice code $\mathcal{C}_{n,\tau}(\lambda)$.

Theorem 1 *The encoding process in Algorithm 1 defines an injective mapping from the message space $\mathcal{M} = \{0,1\}^\mu$ to the lattice code $\mathcal{C}_{n,\tau}(\lambda)$. In particular, when $\mu = \tau n - \frac{n}{4}(k-1)$, the mapping becomes bijective.*

Proof. Denote $\mathcal{V} = \prod_{j=0}^{\frac{n}{2}-1} \mathcal{V}_{2\tau-w_H(j)}$. Based on the definition of ξ_l , it can be observed that the first six steps in Algorithm 1 correspond to partitioning the bit string and converting into a vector $\mathbf{v} \in \mathcal{V}$. Consequently, this procedure defines an injective mapping from the message space $\mathcal{M} = \{0,1\}^\mu$ to the product space $\mathcal{V}$.

Next, we demonstrate that the mapping $\mathbf{v} \mapsto \lambda \cdot (\mathbf{v} \mathbf{W}_n \bmod 2^\tau)$ is injective for $\mathbf{v} \in \mathcal{V}$. Suppose $\lambda \cdot (\mathbf{v}_1 \mathbf{W}_n \bmod 2^\tau) = \lambda \cdot (\mathbf{v}_2 \mathbf{W}_n \bmod 2^\tau)$, which implies that

$(\mathbf{v}_1 - \mathbf{v}_2) \cdot \mathbf{W}_n = 0 \pmod{2^\tau}$. Let $\tilde{\mathbf{v}} = \mathbf{v}_1 - \mathbf{v}_2 = (\tilde{v}_0, \dots, \tilde{v}_{\frac{n}{2}-1})$. We prove by induction that $\tilde{v}_j = 0$ for all j .

Suppose $\tilde{v}_0 = \dots = \tilde{v}_{j-1} = 0$ (this vacuously holds for the base case $j = 0$). We now show that $\tilde{v}_j = 0$. Here we introduce the definition of the ϕ -adic valuation. For any non-zero $x \in \mathbb{G}$, its ϕ -adic valuation, denoted as $\nu_\phi(x)$, is defined as the largest integer α such that ϕ^α divides x (with $\nu_\phi(0) = \infty$).

By Lemma 5 in Appendix A, $\mathbf{W}_n$ is an upper triangular matrix whose j -th diagonal entry is exactly $\phi^{w_H(j)}$. Thus, we obtain $\tilde{v}_j \cdot \phi^{w_H(j)} = 0 \pmod{2^\tau}$, which implies that

$$\nu_\phi(\tilde{v}_j \cdot \phi^{w_H(j)}) = \nu_\phi(\tilde{v}_j) + \nu_\phi(\phi^{w_H(j)}) \geq \nu_\phi(2^\tau) = \nu_\phi((-i\phi^2)^\tau) = 2\tau.$$

Then we have $\nu_\phi(\tilde{v}_j) \geq 2\tau - w_H(j) \geq \max(2\tau - 2\lceil w_H(j)/2 \rceil, 2\tau - 2\lfloor w_H(j)/2 \rfloor - 1)$.

Denote $\tilde{v}_j = a_j + b_j i$. According to the definition of $\mathcal{V}_{2\tau - w_H(j)}$, the difference $\tilde{v}_j$ between two coordinates in $\mathcal{V}_{2\tau - w_H(j)}$ is strictly bounded by

$$|a_j| < 2^{\tau - \lfloor w_H(j)/2 \rfloor} \quad \text{and} \quad |b_j| < 2^{\tau - \lceil w_H(j)/2 \rceil}.$$

From $\nu_\phi(\tilde{v}_j) \geq 2\tau - 2\lceil w_H(j)/2 \rceil$, we have $\phi^{2\tau - 2\lceil w_H(j)/2 \rceil} = (-2i)^{\tau - \lceil w_H(j)/2 \rceil}$ divides $\tilde{v}_j$. Thus, both the real and imaginary parts of $\tilde{v}_j$ must be multiples of $2^{\tau - \lceil w_H(j)/2 \rceil}$. In particular, $b_j = 0 \pmod{2^{\tau - \lceil w_H(j)/2 \rceil}}$. Given the strict bound on $|b_j|$, this immediately implies $b_j = 0$. Now, we know $\tilde{v}_j = a_j$. Since $\nu_\phi(\tilde{v}_j) \geq 2\tau - 2\lfloor w_H(j)/2 \rfloor - 1$, we have $\phi^{2\tau - 2\lfloor w_H(j)/2 \rfloor} = (-2i)^{\tau - \lfloor w_H(j)/2 \rfloor}$ divides $\tilde{v}_j \cdot \phi = a_j + a_j i$. Similarly, we obtain $a_j = 0 \pmod{2^{\tau - \lfloor w_H(j)/2 \rfloor}}$, which leads to $a_j = 0$.

Hence, we conclude that $\tilde{v}_j = 0$ for all j , which proves the injectivity of the encoding process. Moreover, when $\mu = \tau n - \frac{n}{4}(k-1)$, we have $|\mathcal{M}| = |\mathcal{C}_{n,\tau}(\lambda)|$, implying that the mapping is bijective. $\square$

Lattice Decoding Lattice decoding consists of two main sub-processes. The first sub-process involves error correction for a noisy vector $\mathbf{x} = \text{Encode}(\mathbf{m}) + \mathbf{e}$ to recover $\text{Encode}(\mathbf{m})$. This error correction is typically performed using either Maximum Likelihood Decoding (MLD) or Bounded Distance Decoding (BDD).

To the best of our knowledge, the only existing MLD algorithm for Barnes-Wall lattices of arbitrary dimension was proposed by Forney in 1988, which utilizes the trellis representation of BW_n [11]. However, the complexity of this algorithm grows exponentially with the dimension, making it intractable for $n > 32$. Later, Micciancio and Nicolosi [18] proposed the first BDD algorithm for Barnes-Wall lattices that runs in polynomial time. A double-sided version of this decoder was subsequently developed using the squaring construction in [9].

In this paper, we advance the state-of-the-art in BDD for Barnes-Wall lattices. Rather than a straightforward adaptation, we generalize the decoding framework of [9] to propose a novel, scalable BDD algorithm explicitly tailored for $\mathcal{C}_{n,\tau}(\lambda)$, as detailed in Algorithm 2, which significantly enhances both the flexibility and efficiency of the decoding process.

One practical implication of the scalability property is illustrated in lines 6 - 7 of Algorithm 2. Specifically, by scaling $\mathcal{C}_{n/2,\tau}(\lambda)$ to $\mathcal{C}_{n/2,\tau}(2\lambda)$, the computation

Algorithm 2 $\text{BDD}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{x} \in \mathbb{C}^{n/2})$

Input: A target vector $\mathbf{x} \in \mathbb{C}^{n/2}$

Output: A lattice code vector $\mathbf{w} \in \mathcal{C}_{n,\tau}(\lambda)$

```

1: if  $n = 2$  then
2:   return  $\lambda(\lfloor \frac{\mathbf{x}}{\lambda} \rfloor \bmod 2^\tau)$ 
3: else
4:   Divide  $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2)$  such that  $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{C}^{n/4}$ 
5:    $\mathbf{y}_1 := \text{BDD}_{\mathcal{C}_{n/2,\tau}(\lambda)}(\mathbf{x}_1)$ ,  $\mathbf{y}_2 := \text{BDD}_{\mathcal{C}_{n/2,\tau}(\lambda)}(\mathbf{x}_2)$ 
6:    $\mathbf{z}_1 := \frac{1}{2}\text{BDD}_{\mathcal{C}_{n/2,\tau}(2\lambda)}(2\phi^{-1}(\mathbf{x}_2 - \mathbf{y}_1)) (= \text{BDD}_{\mathcal{C}_{n/2,\tau}(\lambda)}(\phi^{-1}(\mathbf{x}_2 - \mathbf{y}_1)))$ 
7:    $\mathbf{z}_2 := \frac{1}{2}\text{BDD}_{\mathcal{C}_{n/2,\tau}(2\lambda)}(2\phi^{-1}(\mathbf{x}_1 - \mathbf{y}_2)) (= \text{BDD}_{\mathcal{C}_{n/2,\tau}(\lambda)}(\phi^{-1}(\mathbf{x}_1 - \mathbf{y}_2)))$ 
8:    $\text{dis}_1 := \|\mathbf{y}_1 - \mathbf{x}_1\|_{2^\tau\lambda,2}^2 + \|\mathbf{y}_1 + \phi\mathbf{z}_1 - \mathbf{x}_2\|_{2^\tau\lambda,2}^2$ 
9:    $\text{dis}_2 := \|\mathbf{y}_2 - \mathbf{x}_2\|_{2^\tau\lambda,2}^2 + \|\mathbf{y}_2 + \phi\mathbf{z}_2 - \mathbf{x}_1\|_{2^\tau\lambda,2}^2$ 
10:  if  $\text{dis}_1 < \text{dis}_2$  then
11:    return  $(\mathbf{y}_1, \mathbf{y}_1 + \phi\mathbf{z}_1)$ 
12:  else
13:    return  $(\mathbf{y}_2 + \phi\mathbf{z}_2, \mathbf{y}_2)$ 
14:  end if
15: end if

```

of $\text{BDD}_{\mathcal{C}_{n/2,\tau}(\lambda)}(\mathbf{x})$ can be reformulated as

$$\text{BDD}_{\mathcal{C}_{n/2,\tau}(\lambda)}(\mathbf{x}) = \frac{1}{2}\text{BDD}_{\mathcal{C}_{n/2,\tau}(2\lambda)}(2\mathbf{x}).$$

This transformation, when combined with the subsequent parameter selection for the cryptographic scheme, ensures that all computations are performed strictly over integers. This avoids operations over dyadic rationals that would otherwise arise from the use of $\phi^{-1} = \frac{1-i}{2}$, thereby eliminating the need for fixed-point or rational arithmetic. Such an integer-only approach is particularly crucial in cryptographic implementations, where it contributes to both efficiency and implementation security.

Lemma 2 *Given a target vector $\mathbf{x}$ satisfying $\text{dist}(\mathbf{x}, \mathcal{C}_{n,\tau}(\lambda)) < r = \sqrt{\frac{n}{8}}\lambda$. Then $\text{BDD}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{x})$ in Algorithm 2 outputs the unique lattice code vector $\mathbf{w} \in \mathcal{C}_{n,\tau}(\lambda)$ such that $\|\mathbf{x} - \mathbf{w}\|_{2^\tau\lambda,2} < r$.*

Proof. According to Lemma 1, the packing radius of $\text{BW}_n(\lambda)$ is $\sqrt{\frac{n}{8}}\lambda = r$. Therefore, if there exists a vector $\mathbf{w} \in \mathcal{C}_{n,\tau}(\lambda)$ satisfying $\|\mathbf{x} - \mathbf{w}\|_{2^\tau\lambda,2} < r$, this ensures that $\mathbf{w}$ is the closest lattice codeword to $\mathbf{x}$, thereby guaranteeing uniqueness. It remains to show that $\text{BDD}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{x})$ indeed corresponds to this unique lattice codeword.

We begin by bisecting $\mathbf{w} = (\mathbf{w}_1, \mathbf{w}_2)$. Since $\mathbf{w} \in \mathcal{C}_{n,\tau}(\lambda)$, it follows from Definition 1 that $\mathbf{w}_1, \mathbf{w}_2, \phi^{-1}(\mathbf{w}_2 - \mathbf{w}_1) \in \mathcal{C}_{n/2,\tau}(\lambda)$. We now establish that $\mathbf{w} = \text{BDD}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{x})$ by induction.

For the base case $n = 2$, it is straightforward to verify that the unique lattice vector in $\mathcal{C}_{2,\tau}(\lambda)$ is given by $\lambda(\left\lfloor \frac{\mathbf{x}}{\lambda} \right\rfloor \bmod 2^\tau)$. For $n = 2^k \geq 4$, since $\|\mathbf{x} - \mathbf{w}\|_{2^\tau\lambda,2}^2 = \|\mathbf{x}_1 - \mathbf{w}_1\|_{2^\tau\lambda,2}^2 + \|\mathbf{x}_2 - \mathbf{w}_2\|_{2^\tau\lambda,2}^2$, at least one of the following inequalities must hold:

$$\|\mathbf{x}_1 - \mathbf{w}_1\|_{2^\tau\lambda,2}^2 < \frac{n\lambda^2}{16} \quad \text{or} \quad \|\mathbf{x}_2 - \mathbf{w}_2\|_{2^\tau\lambda,2}^2 < \frac{n\lambda^2}{16}.$$

- **Case 1:** $\|\mathbf{x}_1 - \mathbf{w}_1\|_{2^\tau\lambda,2}^2 < n\lambda^2/16$. Since $\text{dist}(\mathbf{x}_1, \mathcal{C}_{n/2,\tau}(\lambda)) \leq \|\mathbf{x}_1 - \mathbf{w}_1\|_{2^\tau\lambda,2} < \sqrt{n}\lambda/4 = \rho(\text{BW}_{n/2}(\lambda))$, by induction we have $\mathbf{y}_1 = \mathbf{w}_1$ in line 5 of Algorithm 2. Furthermore, since $\|\mathbf{x}_2 - \mathbf{w}_2\|_{2^\tau\lambda,2}^2 < n\lambda^2/8$, we obtain that $\|\phi^{-1}(\mathbf{x}_2 - \mathbf{w}_2)\|_{2^\tau\lambda,2}^2 < n\lambda^2/16$, which implies that $\|\phi^{-1}(\mathbf{x}_2 - \mathbf{y}_1) - \phi^{-1}(\mathbf{w}_2 - \mathbf{w}_1)\|_{2^\tau\lambda,2}^2 < n\lambda^2/16$. Since $\phi^{-1}(\mathbf{w}_2 - \mathbf{w}_1) \in \mathcal{C}_{n/2,\tau}(\lambda)$, the induction hypothesis ensures that $\mathbf{z}_1 = \phi^{-1}(\mathbf{w}_2 - \mathbf{w}_1)$ in line 6.
- **Case 2:** $\|\mathbf{x}_2 - \mathbf{w}_2\|_{2^\tau\lambda,2}^2 < n\lambda^2/16$. By a similar argument, we conclude that $\mathbf{y}_2 = \mathbf{w}_2$ and $\mathbf{z}_2 = \phi^{-1}(\mathbf{w}_1 - \mathbf{w}_2)$.

Hence, either $(\mathbf{y}_1, \mathbf{y}_1 + \phi\mathbf{z}_1)$ or $(\mathbf{y}_2 + \phi\mathbf{z}_2, \mathbf{y}_2)$ equals to the unique vector $\mathbf{w}$. After comparing their distance to $\mathbf{x}$, the output of Algorithm 2 satisfies $\text{BDD}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{x}) = \mathbf{w}$, completing the proof. $\square$

Algorithm 3 $\text{Decode}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{x} \in \mathbb{C}^{n/2})$

Input: A target vector $\mathbf{x} \in \mathbb{C}^{n/2}$

Output: μ -bit string $\mathbf{m} \in \{0, 1\}^\mu$

- 1: $\mathbf{w} := \text{BDD}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{x})$
 - 2: $\mathbf{v} := \frac{1}{\lambda}\mathbf{w} \cdot \mathbf{W}_n^{-1}$
 - 3: Denote $\mathbf{v} = (v_0, \dots, v_{\frac{n}{2}-1})$, where $v_j = a_j + b_j i$
 - 4: **for** j from 0 to $\frac{n}{2} - 1$ **do**
 - 5: $b'_j := b_j \bmod 2^{\tau - \lceil w_H(j)/2 \rceil}$
 - 6: $a'_j := a_j - (b_j - b'_j) \bmod 2^{\tau - \lfloor w_H(j)/2 \rfloor}$
 - 7: $v'_j := a'_j + b'_j i$
 - 8: $\mathbf{m}_j := \xi_{2^{\tau - w_H(j)}}^{-1}(v'_j)$
 - 9: **end for**
 - 10: Set $\bar{\mathbf{m}} = (\mathbf{m}_0, \mathbf{m}_1, \dots, \mathbf{m}_{\frac{n}{2}-1})$ and define $\mathbf{m}$ as the first μ bits of $\bar{\mathbf{m}}$
 - 11: **return** $\mathbf{m}$
-

The second sub-process serves as the inverse of the encoding procedure, as described in lines 2 to 11 of Algorithm 3. Due to the modulo operation in line 7 of Algorithm 1, it is necessary to adjust the entries of the vector $\mathbf{v}$ to ensure that it belongs to the set $\mathcal{V}$, as defined in the proof of Theorem 1. The following theorem establishes the correctness of the decoding procedure.

Theorem 2 (Correct Decoding Bound) *Let $\mathbf{m}_0 \in \{0, 1\}^\mu$ be a given message, and define $\mathbf{w}_0 := \text{Encode}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{m}_0)$. For any $\mathbf{x} \in \mathbb{C}^{n/2}$, let $\mathbf{m} := \text{Decode}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{x})$. If $\|\mathbf{x} - \mathbf{w}_0\|_{2^\tau\lambda,2} < \sqrt{\frac{n}{8}}\lambda$, then $\mathbf{m} = \mathbf{m}_0$.*

Proof. From the encoding process in Algorithm 1, let $\mathbf{w}_0 = \lambda \cdot (\mathbf{v}_0 \mathbf{W}_n \bmod 2^\tau)$, where $\mathbf{v}_0 \in \mathcal{V}$ corresponds to $\mathbf{m}_0$. Since $\|\mathbf{x} - \mathbf{w}_0\|_{2^\tau \lambda, 2} < \sqrt{\frac{n}{8}} \lambda$, applying Lemma 2 yields $\text{BDD}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{x}) = \mathbf{w}_0$. Thus, when decoding with $\mathbf{x}$ in Algorithm 3, we obtain

$$\mathbf{v} = \frac{1}{\lambda} \text{BDD}_{\mathcal{C}_{n,\tau}(\lambda)}(\mathbf{x}) \cdot \mathbf{W}_n^{-1} = (\mathbf{v}_0 \mathbf{W}_n \bmod 2^\tau) \cdot \mathbf{W}_n^{-1}. \quad (2)$$

Denote $\mathbf{v} = (v_0, \dots, v_{\frac{n}{2}-1})$ and $\mathbf{v}' = (v'_0, \dots, v'_{\frac{n}{2}-1})$ in Algorithm 3, where $\mathbf{v}' \in \mathcal{V}$ corresponds to $\mathbf{m}$. To complete the proof, it suffices to show that $\mathbf{v}' = \mathbf{v}_0$, since this naturally implies $\mathbf{m} = \mathbf{m}_0$ according to the definition of ξ_l .

Recall the ϕ -adic valuation $\nu_\phi(\cdot)$ introduced in the proof of Theorem 1. We first demonstrate that lines 4 to 9 of Algorithm 3 ensure that $\phi^{2\tau - w_H(j)}$ divides $(v_j - v'_j)$, which is equivalent to showing $\nu_\phi(v_j - v'_j) \geq 2\tau - w_H(j)$. Since $v_j - v'_j = (a_j - a'_j - (b_j - b'_j)) + (b_j - b'_j)\phi$ and $\nu_\phi(2) = \nu_\phi(-i\phi^2) = 2$, by the definitions of a'_j and b'_j , we obtain

$$\begin{aligned} \nu_\phi(a_j - a'_j - (b_j - b'_j)) &\geq \nu_\phi(2^{\tau - \lfloor w_H(j)/2 \rfloor}) = 2\tau - 2\lfloor w_H(j)/2 \rfloor, \\ \nu_\phi((b_j - b'_j)\phi) &\geq \nu_\phi(2^{\tau - \lceil w_H(j)/2 \rceil}) + \nu_\phi(\phi) = 2\tau - 2\lceil w_H(j)/2 \rceil + 1. \end{aligned}$$

By the triangle inequality, it follows that

$$\nu_\phi(v_j - v'_j) \geq \min(2\tau - 2\lfloor w_H(j)/2 \rfloor, 2\tau - 2\lceil w_H(j)/2 \rceil + 1) = 2\tau - w_H(j),$$

which implies that $\phi^{2\tau - w_H(j)} \mid (v_j - v'_j)$, as desired.

Then, by Lemma 5 in Appendix A, each entry in the j -th row of $\mathbf{W}_n$ is divisible by $\phi^{w_H(j)}$. Thus each entry of $(\mathbf{v} - \mathbf{v}') \cdot \mathbf{W}_n$ is divisible by $\phi^{2\tau}$, which implies $(\mathbf{v} - \mathbf{v}') \cdot \mathbf{W}_n = 0 \bmod 2^\tau$. Therefore, from (2) we deduce $\mathbf{v}' \mathbf{W}_n = \mathbf{v}_0 \mathbf{W}_n \bmod 2^\tau$. By the injectivity indicated in Theorem 1, it follows that $\mathbf{v}' = \mathbf{v}_0$, which completes the proof. $\square$

4 Construction and Analysis

4.1 The IND-CPA-secure PKE

Let κ, l, d_u, d_v be positive integer parameters. Let $n = 2^k$ and τ be the parameters of the nested lattices $\mathcal{C}_{n,\tau}(\lambda)$. The message space is defined as $\mathcal{M} = \{0, 1\}^{\frac{N}{n} \mu_0}$, where $\mu_0 = \tau n - \frac{n}{4}(k-1)$. Each message $m \in \mathcal{M}$ is represented as a polynomial in $\mathcal{R}$ with binary coefficients. Our public-key encryption scheme BW-PKE is described in Figure 2.

We use deterministic seed-expansion algorithms to generate the objects used by BW-PKE. The algorithm `SampleA` expands a seed $\rho \in \{0, 1\}^\kappa$ into a matrix $\mathbf{A} \in \mathcal{R}_q^{l \times l}$. The algorithm `SampleSE $_\eta$` expands a seed into a pair of vectors in $\mathcal{B}_\eta^l \times \mathcal{B}_\eta^l$. The algorithms `SampleR $_\eta$` and `SampleE $_\eta$` expand a seed into, respectively, a vector in $\mathcal{B}_\eta^l$ and a pair in $\mathcal{B}_\eta^l \times \mathcal{B}_\eta$. Unless otherwise specified, all seeds and key strings used by BW-PKE and BW-KEM have length κ . In the security analysis,

these function can be treated as pseudorandom generator (PRG) in the standard model, or modeled as random oracle to explain the randomness of the output distribution. We assume for simplicity that the output distributions of seed-expansion algorithms are ideal.

The scalable lattice coding introduced in Section 3.2 is primarily realized through the PolyEncode and PolyDecode procedures. This newly designed error-correction mechanism constitutes the main distinction between BW-PKE and the K-PKE used in ML-KEM [22], and it fully demonstrates the flexibility and efficiency of scalability. In the PolyEncode algorithm, the scaling factor is set to $\frac{q}{2^\tau}$. The input message is divided into $d = \frac{N}{n}$ blocks, each containing μ_0 bits, and each block is encoded via $\text{Encode}_{\mathcal{C}_{n,\tau}(\frac{q}{2^\tau})}$ into n coefficients spaced d positions apart, as shown in line 6 of PolyEncode. The rationale behind this encoding strategy will be further discussed in Section 5.1.

In contrast, the decoding procedure first performs a modulus-switching operation on the target polynomial $v' - \mathbf{s}^T \mathbf{u}'$ (line 3 of BW-PKE.Dec), thereby adopting a different scaling factor $\frac{\hat{q}}{2^\tau}$ in the subsequent PolyDecode algorithm, where $\hat{q} = 2^{\lceil \log_2 q \rceil}$. This choice of a power-of-two scaling factor simplifies the arithmetic in the decoding process by enabling efficient bit-shifts and eliminating reliance on fixed-point arithmetic, thus significantly improving both computational efficiency and implementation security. The target polynomial is then similarly divided into d blocks, each consisting of n coefficients, and decoded via $\text{Decode}_{\mathcal{C}_{n,\tau}(\frac{\hat{q}}{2^\tau})}$.

BW-PKE.KeyGen()	BW-PKE.Enc(pk = (t, ρ), m; r ∈ {0, 1}^κ)	BW-PKE.Dec(sk = s, c = (u, v))
1: $\rho, \sigma \xleftarrow{\$} \{0, 1\}^\kappa$	1: $\mathbf{A} \sim \mathcal{R}_q^{l \times l} := \text{SampleA}(\rho)$	1: $\mathbf{u}' := \text{Decompress}_{2^d u, q}(\mathbf{u})$
2: $\mathbf{A} \sim \mathcal{R}_q^{l \times l} := \text{SampleA}(\rho)$	2: $\mathbf{r} \sim \beta_{\eta_1}^l := \text{SampleR}_{\eta_1}(r)$	2: $v' := \text{Decompress}_{2^d v, q}(v)$
3: $(\mathbf{s}, \mathbf{e}) \sim \beta_{\eta_1}^l \times \beta_{\eta_1}^l := \text{SampleSE}_{\eta_1}(\sigma)$	3: $(\mathbf{e}_1, \mathbf{e}_2) \sim \beta_{\eta_2}^l \times \beta_{\eta_2}^l := \text{SampleE}_{\eta_2}(r)$	3: $w := \left\lfloor \frac{\hat{q}}{q} (v' - \mathbf{s}^T \mathbf{u}') \right\rfloor$
4: $\mathbf{t} := \mathbf{A}\mathbf{s} + \mathbf{e}$	4: $\mathbf{u}_0 := \mathbf{A}^T \mathbf{r} + \mathbf{e}_1$	4: $m' := \text{PolyDecode}(w \bmod \pm \hat{q})$
5: return (pk := (t, ρ), sk := s)	5: $v_0 := \mathbf{t}^T \mathbf{r} + \mathbf{e}_2$	5: return m'
	6: $\mathbf{u} := \text{Compress}_{q, 2^d u}(\mathbf{u}_0)$	
	7: $v := \text{Compress}_{q, 2^d v}(v_0 + \lfloor \text{PolyEncode}(m) \rfloor)$	
	8: return c := (u, v)	
PolyEncode($m = \sum_{j=0}^{\frac{N}{n}-1} m_j x^j \in \mathcal{M}$)	PolyDecode($w = \sum_{j=0}^{\frac{N}{n}-1} w_j x^j \in \mathcal{R}_{\hat{q}}$)	
1: for j from 0 to $\frac{N}{n} - 1$ do	1: for j from 0 to $\frac{N}{n} - 1$ do	
2: $\mathbf{m}_j := (m_{\mu_0 j}, m_{\mu_0 j+1}, \dots, m_{\mu_0 j+\mu_0-1}) \in \{0, 1\}^{\mu_0}$	2: $\mathbf{a}_j := (w_j, w_{j+2d}, \dots, w_{j+(n-2)d}) \in \mathbb{R}^{n/2}$	
3: $\mathbf{w}_j := \text{Encode}_{\mathcal{C}_{n,\tau}(q/2^\tau)}(\mathbf{m}_j) \in \mathbb{C}^{n/2}$	3: $\mathbf{b}_j := (w_{j+d}, w_{j+3d}, \dots, w_{j+(n-1)d}) \in \mathbb{R}^{n/2}$	
4: Decompose $\mathbf{w}_j$ into $\mathbf{w}_j = \mathbf{a}_j + \mathbf{b}_j i$, where $\mathbf{a}_j, \mathbf{b}_j \in \mathbb{R}^{n/2}$	4: $\mathbf{x}_j := \mathbf{a}_j + \mathbf{b}_j i \in \mathbb{C}^{n/2}$	
5: $(a_{j,0}, a_{j,1}, \dots, a_{j,n/2-1}) := \mathbf{a}_j, (b_{j,0}, b_{j,1}, \dots, b_{j,n/2-1}) := \mathbf{b}_j$	5: $\mathbf{m}_j := \text{Decode}_{\mathcal{C}_{n,\tau}(q/2^\tau)}(\mathbf{x}_j) \in \{0, 1\}^{\mu_0}$	
6: $(w_j, w_{j+d}, \dots, w_{j+(n-1)d}) := (a_{j,0}, b_{j,0}, \dots, a_{j,n/2-1}, b_{j,n/2-1})$	6: $(m_{\mu_0 j}, m_{\mu_0 j+1}, \dots, m_{\mu_0 j+\mu_0-1}) := \mathbf{m}_j$	
7: endfor	7: endfor	
8: $w := \sum_{j=0}^{\frac{N}{n}-1} w_j x^j$	8: $m := \sum_{j=0}^{\frac{N}{n}-1} m_j x^j \in \mathcal{M}$	
9: return w	9: return m	

Fig. 2. The description of BW-PKE.

Correctness of the PKE Scheme We analyze the correctness of BW-PKE by evaluating its decryption error probability. To this end, we first establish the following lemma, which illustrates the flexibility gained from employing distinct scaling factors.

Lemma 3 *It holds that $w \bmod^{\pm \hat{q}} = \frac{\hat{q}}{q}(\frac{q}{\hat{q}}w \bmod^{\pm q})$. Consequently, we have $\text{PolyDecode}_{\mathcal{C}_{n,\tau}(\frac{\hat{q}}{2^\tau})}(w \bmod^{\pm \hat{q}}) = \text{PolyDecode}_{\mathcal{C}_{n,\tau}(\frac{q}{2^\tau})}(\frac{q}{\hat{q}}w \bmod^{\pm q})$.*

Proof. Since the centered modular reduction $\frac{q}{\hat{q}}w \bmod^{\pm q}$ implies the existence of a polynomial $\theta \in \mathbb{Z}[x]/(x^N + 1)$ with integer coefficients such that

$$\frac{q}{\hat{q}}w \bmod^{\pm q} = \frac{q}{\hat{q}}w + q\theta,$$

each component of $\frac{q}{\hat{q}}w + q\theta$ lies in the interval $[-\frac{q}{2}, \frac{q}{2})$. Multiplying both sides by $\frac{\hat{q}}{q}$ yields that every component of $w + \hat{q}\theta$ lies in $[-\frac{\hat{q}}{2}, \frac{\hat{q}}{2})$, and hence

$$\begin{aligned} w \bmod^{\pm \hat{q}} &= w + \hat{q}\theta \\ &= \frac{\hat{q}}{q} \left(\frac{q}{\hat{q}}w + q\theta \right) \\ &= \frac{\hat{q}}{q} \left(\frac{q}{\hat{q}}w \bmod^{\pm q} \right). \end{aligned}$$

Now consider the decoding operation. Scaling the lattice code $\mathcal{C}_{n,\tau}(\frac{\hat{q}}{2^\tau})$ and the input $w \bmod^{\pm \hat{q}}$ by the same factor $\frac{q}{\hat{q}}$ gives

$$\begin{aligned} m' &= \text{PolyDecode}_{\mathcal{C}_{n,\tau}(\frac{\hat{q}}{2^\tau})}(w \bmod^{\pm \hat{q}}) \\ &= \text{PolyDecode}_{\mathcal{C}_{n,\tau}(\frac{\hat{q}}{q} \cdot \frac{q}{2^\tau})} \left(\frac{\hat{q}}{q} \left(\frac{q}{\hat{q}}w \bmod^{\pm q} \right) \right) \\ &= \text{PolyDecode}_{\mathcal{C}_{n,\tau}(\frac{q}{2^\tau})} \left(\frac{q}{\hat{q}}w \bmod^{\pm q} \right). \end{aligned}$$

This completes the proof. $\square$

Based on Lemma 3, we now present the following theorem that formalizes the correctness guarantee of BW-PKE.

Theorem 3 *Let $\mathbf{s}, \mathbf{e}, \mathbf{r}, \mathbf{e}_1, \mathbf{e}_2$ be random variables distributed as specified in Figure 2. Let $\boldsymbol{\epsilon}_{\mathbf{u}} \leftarrow \phi_{d_{\mathbf{u}}}^l$ and $\epsilon_v \leftarrow \phi_{d_v}$, where the distribution ϕ_d^m over $\mathcal{R}^m$ is defined as follows: sample $\mathbf{x} \xleftarrow{\$} \mathcal{R}^m$ and output $(\mathbf{x} - \text{Decompress}_{2^d,q}(\text{Compress}_{q,2^d}(\mathbf{x}))) \bmod^{\pm q}$.*

For a polynomial f , we define its i -th block as $(f)_i = (f_i, f_{i+d}, f_{i+2d}, \dots, f_{i+(n-1)d})$ for $i \in \{0, 1, \dots, d-1\}$. That is, each block comprises n coefficients spaced exactly $d = \frac{N}{n}$ positions apart. Let Err be the error polynomial $\mathbf{e}^T \mathbf{r} - \mathbf{s}^T(\mathbf{e}_1 + \boldsymbol{\epsilon}_{\mathbf{u}}) + \mathbf{e}_2 + \epsilon_v$. Denote

$$\delta = 1 - \Pr[\|(\text{Err})_i\|_{q,2} < r_0, \forall i],$$

where $r_0 = \frac{q\sqrt{2n}}{2^{\tau+2}} - \frac{\sqrt{n}}{2} \left(\frac{q}{\hat{q}} + 1 \right)$. Then BW-PKE is $(1 - \delta)$ -correct.

Proof. According to the BW-PKE.Dec procedure described in Figure 2, the value of $\mathbf{u}'$ is given by

$$\begin{aligned}\mathbf{u}' &= \text{Decompress}_{2^{d_u}, q}(\text{Compress}_{q, 2^{d_u}}(\mathbf{A}^T \mathbf{r} + \mathbf{e}_1)) \\ &= \mathbf{A}^T \mathbf{r} + \mathbf{e}_1 + \boldsymbol{\epsilon}_u,\end{aligned}$$

for some $\boldsymbol{\epsilon}_u \in \mathcal{R}^l$. Similarly, the value of v' is computed as

$$\begin{aligned}v' &= \text{Decompress}_{2^{d_v}, q}(\text{Compress}_{q, 2^{d_v}}(\mathbf{t}^T \mathbf{r} + e_2 + \lfloor \text{PolyEncode}(m) \rfloor)) \\ &= (\mathbf{A}\mathbf{s} + \mathbf{e})^T \mathbf{r} + e_2 + \lfloor \text{PolyEncode}(m) \rfloor + \epsilon_v,\end{aligned}$$

for some $\epsilon_v \in \mathcal{R}$. We may safely assume that $\boldsymbol{\epsilon}_u$ and ϵ_v follow the distributions $\phi_{d_u}^l$ and ϕ_{d_v} , respectively, since they both take the form

$$(\mathbf{x} - \text{Decompress}_{2^d, q}(\text{Compress}_{q, 2^d}(\mathbf{x}))) \bmod^{\pm} q,$$

where $\mathbf{x}$ is pseudorandom under the hardness assumption of the Module-LWE problem.

Denote $\epsilon_1 = \left\lfloor \frac{\hat{q}}{q}(v' - \mathbf{s}^T \mathbf{u}') \right\rfloor - \frac{\hat{q}}{q}(v' - \mathbf{s}^T \mathbf{u}')$, and $\epsilon_2 = \lfloor \text{PolyEncode}(m) \rfloor - \text{PolyEncode}(m)$. From Lemma 3, it follows that

$$m' = \text{PolyDecode}_{\mathcal{C}_{n, \tau}(\frac{q}{2^\tau})} \left(\frac{q}{\hat{q}} \left\lfloor \frac{\hat{q}}{q}(v' - \mathbf{s}^T \mathbf{u}') \right\rfloor \bmod^{\pm} q \right).$$

We then derive

$$\begin{aligned}\frac{q}{\hat{q}} \left\lfloor \frac{\hat{q}}{q}(v' - \mathbf{s}^T \mathbf{u}') \right\rfloor &= v' - \mathbf{s}^T \mathbf{u}' + \frac{q}{\hat{q}} \epsilon_1 \\ &= (\mathbf{A}\mathbf{s} + \mathbf{e})^T \mathbf{r} + e_2 + \lfloor \text{PolyEncode}(m) \rfloor + \epsilon_v - \mathbf{s}^T (\mathbf{A}^T \mathbf{r} + \mathbf{e}_1 + \boldsymbol{\epsilon}_u) + \frac{q}{\hat{q}} \epsilon_1 \\ &= \text{PolyEncode}(m) + \mathbf{e}^T \mathbf{r} - \mathbf{s}^T (\mathbf{e}_1 + \boldsymbol{\epsilon}_u) + e_2 + \epsilon_v + \frac{q}{\hat{q}} \epsilon_1 + \epsilon_2.\end{aligned}$$

Every n coefficients spaced $d = \frac{N}{n}$ positions apart of $\text{PolyEncode}(m)$ (referred to as a *block*) essentially correspond to a lattice code vector in $\mathcal{C}_{n, \tau}(\frac{q}{2^\tau})$. The deviation from the target lattice point is then bounded by

$$\begin{aligned}&\|(\mathbf{e}^T \mathbf{r} - \mathbf{s}^T (\mathbf{e}_1 + \boldsymbol{\epsilon}_u) + e_2 + \epsilon_v)_i\|_{q, 2} + \frac{q}{\hat{q}} \|(\epsilon_1)_i\|_{q, 2} + \|(\epsilon_2)_i\|_{q, 2} \\ &\leq \|(\mathbf{e}^T \mathbf{r} - \mathbf{s}^T (\mathbf{e}_1 + \boldsymbol{\epsilon}_u) + e_2 + \epsilon_v)_i\|_{q, 2} + \frac{\sqrt{n}}{2} \left(\frac{q}{\hat{q}} + 1 \right),\end{aligned}$$

since $(\epsilon_1)_i, (\epsilon_2)_i \in (-\frac{1}{2}, \frac{1}{2}]^n$.

According to Theorem 2, decoding succeeds whenever the above deviation is smaller than the packing radius $\frac{q\sqrt{2n}}{2^{\tau+2}}$. Hence, the decryption error probability of BW-PKE is

$$1 - \Pr \left[\left\| (\mathbf{e}^T \mathbf{r} - \mathbf{s}^T \mathbf{e}_1 - \mathbf{s}^T \mathbf{e}_u + e_2 + \epsilon_v)_i \right\|_{q,2} < r_0, \forall i \right], \quad (3)$$

where $r_0 = \frac{q\sqrt{2n}}{2^{\tau+2}} - \frac{\sqrt{n}}{2}(\frac{q}{q} + 1)$. $\square$

The above provides only the formal statement of correctness. A more comprehensive analysis, including the evaluation methodology for estimating the decryption error rate under concrete parameter settings, will be provided in Section 5.

Security of the PKE Scheme The following theorem establishes the IND-CPA security of BW-PKE under the MLWE assumption.

Theorem 4 (IND-CPA security of BW-PKE) *For any adversary $\mathcal{A}$ against the IND-CPA security of BW-PKE, there exists an adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{BW-PKE}}^{\text{IND-CPA}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{l+1,l,\beta_\eta}^{\text{MLWE}}(\mathcal{B}).$$

Proof. Let $\mathcal{A}$ be an adversary against the IND-CPA security, and consider the games defined in Figure 3.

GAMES $G_0^{\mathcal{A}} - G_2^{\mathcal{A}}$	
1 :	$(\text{pk}, \text{sk}) \leftarrow \text{BW-PKE.KeyGen}()$
2 :	$\text{pk} := (\rho \xleftarrow{\$} \{0,1\}^\kappa, \mathbf{t} \xleftarrow{\$} \mathcal{R}_q^l) \quad \triangleright G_1^{\mathcal{A}} - G_2^{\mathcal{A}}$
3 :	$(m_0, m_1) \leftarrow \mathcal{A}(\text{pk})$
4 :	$b \xleftarrow{\$} \{0,1\}$
5 :	$(\mathbf{u}, v) \leftarrow \text{BW-PKE.Enc}(\text{pk}, m_b)$
6 :	$\mathbf{u}_0 \xleftarrow{\$} \mathcal{R}_q^l, v_0 \xleftarrow{\$} \mathcal{R}_q \quad \triangleright G_2^{\mathcal{A}}$
7 :	$\mathbf{u} := \text{Compress}_{q,p}(\mathbf{u}_0) \quad \triangleright G_2^{\mathcal{A}}$
8 :	$v := \text{Compress}_{q,g}(v_0 + \lfloor \text{PolyEncode}(m_b) \rfloor) \quad \triangleright G_2^{\mathcal{A}}$
9 :	$b' \leftarrow \mathcal{A}(\mathbf{u}, v)$
10 :	return $\llbracket b' = b \rrbracket$

Fig. 3. Games for the proof of Theorem 4.

GAME $G_0^{\mathcal{A}}$. This is the original $\text{IND-CPA}_{\text{PKE}}^{\mathcal{A}}$ game, thus we have

$$\text{Adv}_{\text{BW-PKE}}^{\text{IND-CPA}}(\mathcal{A}) = \left| \Pr [G_0^{\mathcal{A}} \Rightarrow 1] - \frac{1}{2} \right|.$$

GAME G_1^A . This game is identical to G_0^A except that, in key generation, $\mathbf{t}$ is sampled uniformly at random from $\mathcal{R}_q^l$, i.e., $\mathbf{t} \xleftarrow{\$} \mathcal{R}_q^l$. As a result, $(\mathbf{A}, \mathbf{t})$ becomes a uniformly random tuple, in contrast to G_0^A , where it corresponds to an MLWE sample. An adversary capable of distinguishing between games G_1^A and G_0^A can therefore solve the MLWE problem on this input. Thus, there exists an adversary $\mathcal{B}$ such that

$$|\Pr[G_1^A \Rightarrow 1] - \Pr[G_0^A \Rightarrow 1]| \leq \text{Adv}_{l,l,\beta_\eta}^{\text{MLWE}}(\mathcal{B}) \leq \text{Adv}_{l+1,l,\beta_\eta}^{\text{MLWE}}(\mathcal{B}).$$

GAME G_2^A . In game G_1^A , $\begin{pmatrix} \mathbf{A}^T \\ \mathbf{t}^T \end{pmatrix}$ is uniformly distributed over $\mathcal{R}_q^{(l+1) \times l}$ and $\left(\begin{pmatrix} \mathbf{A}^T \\ \mathbf{t}^T \end{pmatrix}, \begin{pmatrix} \mathbf{u}_0 \\ v_0 \end{pmatrix}\right)$ forms an MLWE sample, which is replaced with a uniformly random tuple in game G_2^A . Therefore, an adversary that distinguishes between G_2^A and G_1^A also solves the MLWE problem for this instance. Accordingly, there exists an adversary $\mathcal{B}$ such that

$$|\Pr[G_2^A \Rightarrow 1] - \Pr[G_1^A \Rightarrow 1]| \leq \text{Adv}_{l+1,l,\beta_\eta}^{\text{MLWE}}(\mathcal{B}).$$

In game G_2^A , according to the procedures described in Figure 2, the message m_b is evenly divided into $\frac{N}{n}$ blocks during the encryption process. Each block is then encoded as a lattice codeword in $\mathcal{C}_{n,\tau}(\frac{q}{2^\tau})$ via the PolyEncode procedure. Let $m_b^{(i)}$ denote the i -th block of m_b , which is encoded as $w^{(i)}$, a corresponding lattice codeword. This encoding yields the i -th component of the ciphertext v , denoted as $v^{(i)}$. Since $v^{(i)}$ is computed solely based on $m_b^{(i)}$ and is independent of the remaining blocks of m_b , we aim to show that $v^{(i)}$ is, in fact, independent of $m_b^{(i)}$ as well.

For any i and any fixed $m_b^{(i)}$, the value of $\lfloor w^{(i)} \rfloor$ is deterministically computed. Since v_0 is uniformly distributed in $\mathcal{R}_q$, its i -th component $v_0^{(i)}$ is a uniformly random vector in $\mathbb{Z}_q^n$, and the same holds for $v_0^{(i)} + \lfloor w^{(i)} \rfloor$. Consequently, $v^{(i)}$ is subject to the distribution $\lfloor \frac{q}{q} \tilde{v}^{(i)} \rfloor$, where $\tilde{v}^{(i)}$ corresponds to a uniformly random vector over $\mathbb{Z}_q^n$. This implies that $v^{(i)}$ is independent of $m_b^{(i)}$. Therefore, each component $v^{(i)}$ reveals no information about $m_b^{(i)}$ for all $i \in \{0, 1, \dots, \frac{N}{n} - 1\}$. Hence, we conclude that $\Pr[G_2^A \Rightarrow 1] = \frac{1}{2}$.

Combining the above probability bounds completes the proof. $\square$

4.2 The IND-CCA-secure KEM

We construct our KEM scheme using $\text{FO}_{\text{ID}(\text{pk}),m}^\chi$, a variant of the Fujisaki-Okamoto (FO) transformation [12, 13], designed to achieve strengthened IND-CCA security in the multi-user setting [10], which naturally ensures security in a single-user scenario.

Let $\gamma \in \mathbb{N}$ be integer and $\mathcal{PK}$ denote the public key space. Let $\text{ID} : \mathcal{PK} \rightarrow \{0, 1\}^\gamma$ be a fixed-output-length function. We use random oracles $\text{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa \times \{0, 1\}^\kappa$ and $\text{H}_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$. Thus the encapsulated key, the encryption randomness derived by H , and the implicit-rejection key are all κ -bit strings. The key encapsulation mechanism BW-KEM is specified in Figure 4.

BW-KEM.KeyGen()	BW-KEM.Encaps(pk)	BW-KEM.Decaps((sk, z), c)
1 : $(\text{pk}, \text{sk}) \leftarrow \text{BW-PKE.KeyGen}()$	1 : $m \xleftarrow{\$} \mathcal{M}$	1 : $m' := \text{PKE.Dec}(\text{sk}, c)$
2 : $z \xleftarrow{\$} \{0, 1\}^\kappa$	2 : $(K, r) := \text{H}(\text{ID}(\text{pk}), m)$	2 : $(K', r') := \text{H}(\text{ID}(\text{pk}), m')$
3 : return $(\text{pk}' := \text{pk}, \text{sk}' := (\text{sk}, z))$	3 : $c \leftarrow \text{BW-PKE.Enc}(\text{pk}, m; r)$	3 : $\tilde{K} := \text{H}_1(\text{ID}(\text{pk}), z, c)$
	4 : return (c, K)	4 : if $m' = \perp$ or $\text{BW-PKE.Enc}(\text{pk}, m'; r') \neq c$
		5 : return $\tilde{K}$
		6 : else return K'

Fig. 4. The description of BW-KEM.

Correctness of the KEM Scheme It is evident that if BW-PKE is $(1 - \delta)$ -correct and H is modeled as a random oracle, then BW-KEM is $(1 - \delta)$ -correct.

Security of the KEM Scheme Since our KEM scheme is obtained by applying the $\text{FO}_{\text{ID}(\text{pk}), m}^\ell$ transformation, we leverage the results from [10] to establish the following security theorem. Specifically, we show that BW-KEM achieves IND-CCA security in both the Random Oracle Model (ROM) [3] and the Quantum Random Oracle Model (QROM) [5].

Theorem 5 (IND-CCA security in the (Q)ROM [10]) *Let ℓ be the min-entropy of $\text{ID}(\text{pk})$, i.e., $\ell = \text{H}_\infty(\text{ID}(\text{pk}))$. For any (quantum) adversary $\mathcal{A}$, making at most q_D decapsulation queries, q_H (Q)RO queries, against the IND-CCA security of BW-KEM, there exists (quantum) adversary $\mathcal{B}$ against the IND-CPA security of BW-PKE (with roughly the same running time), such that:*

– In the ROM, it holds that $\text{Adv}_{\text{BW-KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq$

$$2\text{Adv}_{\text{BW-PKE}}^{\text{IND-CPA}}(\mathcal{B}) + \frac{2(q_H + 1)}{|\mathcal{M}|} + \frac{q_H}{2^\kappa} + (q_H + q_D)\delta + \frac{1}{2^\ell};$$

– In the QROM, it holds that $\text{Adv}_{\text{BW-KEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq$

$$2\sqrt{q_{HD}\text{Adv}_{\text{BW-PKE}}^{\text{IND-CPA}}(\mathcal{B})} + \frac{4q_{HD}}{\sqrt{|\mathcal{M}|}} + \frac{4(q_H + 1)}{\sqrt{2^\kappa}} + 16q_{HD}^2\delta + \frac{1}{|\mathcal{M}|} + \frac{1}{2^\ell},$$

where $q_{HD} := q_H + q_D + 1$.

5 Analyzing the DFR without Independence Assumption

Recalling Theorem 3, the decryption failure rate (DFR) of BW-KEM is expressed as $\delta = 1 - \Pr[\|(\text{Err})_i\|_{q,2} < r_0, \forall i]$, where the noise polynomial is defined as $\text{Err} = \mathbf{e}^T \mathbf{r} - \mathbf{s}^T(\mathbf{e}_1 + \mathbf{e}_u) + e_2 + e_v$. It is crucial to note that the coefficients arising from the products $\mathbf{e}^T \mathbf{r}$ and $\mathbf{s}^T(\mathbf{e}_1 + \mathbf{e}_u)$ are inherently dependent. When estimating the failure rate, particularly in PKE schemes such as [16, 23] where error-correcting codes are employed, it is commonly assumed that the errors at different positions are independent. Ignoring this dependence may lead to a substantial underestimation of the DFR. To mitigate this issue and ensure a reliable evaluation of the DFR and concrete security, we therefore develop in this section a methodology for estimating the DFR *without* relying on any independence assumptions among the coefficients of the noise polynomial.

5.1 The Joint Distribution of Coefficients in the Noise Polynomial

The successful decoding probability of the i -th block, denoted by $\Pr[\|(\text{Err})_i\|_{q,2} < r_0]$, depends on the joint distribution of the n coefficients within that block. In this section, we characterize the n -dimensional noise distribution by decomposing its joint structure into a sum of simpler and mutually independent components. The main difficulty lies in analyzing the product terms $\mathbf{e}^T \mathbf{r}$ and $\mathbf{s}^T(\mathbf{e}_1 + \mathbf{e}_u)$. As a starting point, we establish the following lemma, which considers the fundamental case of a single polynomial multiplication.

Lemma 4 *Let $N = 2^{k_0}$ and $n = 2^k$ be powers of two with $k_0 > k > 1$. Suppose $f, g \in \mathcal{R}_q = \mathbb{Z}_q[x]/(x^N + 1)$ have coefficients drawn independently from distributions ψ_f and ψ_g , respectively. Let $h = f \cdot g \in \mathcal{R}_q$. For each $i \in \{0, 1, \dots, d-1\}$ with $d = N/n$, define the partial coefficient vector of h*

$$\mathcal{H}_i = (h_i, h_{i+d}, h_{i+2d}, \dots, h_{i+(n-1)d}),$$

where h_{i+jd} denotes the $(i+jd)$ -th coefficient of h for $j = 0, 1, \dots, n-1$.

Similarly, let $a, b \in \mathbb{Z}_q[y]/(y^n + 1)$ have coefficients drawn independently from ψ_a and ψ_b , respectively. Let $c = a \cdot b \in \mathbb{Z}_q[y]/(y^n + 1)$, and denote by $\text{Prod}_{\psi_a, \psi_b}$ the joint distribution of the coefficient vector $(c_0, c_1, \dots, c_{n-1})$.

If ψ_f is symmetric over $\mathbb{Z}_q$, then each $\mathcal{H}_i$ shares the same distribution and can be expressed as the sum of d independent random variables following $\text{Prod}_{\psi_f, \psi_g}$, namely,

$$\mathcal{H}_i = \sum_{j=0}^{d-1} \mathcal{J}_j, \quad \text{where} \quad \{\mathcal{J}_j\}_{j=0}^{d-1} \stackrel{\text{i.i.d.}}{\sim} \text{Prod}_{\psi_f, \psi_g}.$$

Proof. We begin by establishing the ring isomorphism between $\mathbb{Z}_q[x]/(x^N + 1)$ and $[\mathbb{Z}_q[y]/(y^n + 1)][x]/(x^d - y)$. We define the mapping

$$\begin{aligned} \mathcal{I} : \mathbb{Z}_q[x]/(x^N + 1) &\rightarrow [\mathbb{Z}_q[y]/(y^n + 1)][x]/(x^d - y) \\ \omega(x) &\mapsto \sum_{i=0}^{d-1} \omega^{(i)}(y)x^i, \end{aligned}$$

where $\omega(x) = \omega_0 + \omega_1 x + \cdots + \omega_{N-1} x^{N-1} \in \mathbb{Z}_q[x]/(x^N + 1)$ and $\omega^{(i)}(y) = \sum_{j=0}^{n-1} \omega_{i+jd} y^j \in \mathbb{Z}_q[y]/(y^n + 1)$ for $i = 0, 1, \dots, d-1$. This mapping groups the coefficients of $\omega(x)$ according to their residue classes modulo d , and thereby produces an element of $[\mathbb{Z}_q[y]/(y^n + 1)][x]/(x^d - y)$.

To prove that $\mathcal{I}$ is a ring isomorphism, we rely on a standard argument from commutative ring theory. Consider the bivariate polynomial ring $\mathbb{Z}_q[x, y]$ and the ideal $I = \langle y^n + 1, x^d - y \rangle$. The quotient ring $\mathbb{Z}_q[x, y]/I$ can be evaluated in two equivalent ways. First, eliminating y via the relation $y = x^d$ transforms the generator $y^n + 1$ into $(x^d)^n + 1 = x^N + 1$, which yields the quotient ring $\mathbb{Z}_q[x]/(x^N + 1)$. Alternatively, one may first quotient by $y^n + 1$, obtaining $\mathbb{Z}_q[y]/(y^n + 1)$, and then impose the relation $x^d - y$, resulting in the ring $[\mathbb{Z}_q[y]/(y^n + 1)][x]/(x^d - y)$. Hence, we obtain the natural ring isomorphism:

$$\mathbb{Z}_q[x]/(x^N + 1) \cong [\mathbb{Z}_q[y]/(y^n + 1)][x]/(x^d - y).$$

The mapping $\mathcal{I}$ explicitly realizes this isomorphism by substituting $y = x^d$ and collecting monomials according to their degree modulo d .

Let $f(x) = \sum_{r=0}^{N-1} f_r x^r$ and $g(x) = \sum_{r=0}^{N-1} g_r x^r$ be elements in $\mathbb{Z}_q[x]/(x^N + 1)$. Since $\mathcal{I}$ is a ring isomorphism, it strictly preserves multiplication, i.e., $\mathcal{I}(f(x) \cdot g(x)) = \mathcal{I}(f(x)) \cdot \mathcal{I}(g(x))$. Let $h(x) = f(x) \cdot g(x) \in \mathbb{Z}_q[x]/(x^N + 1)$. Through the isomorphism, polynomial multiplication in $\mathbb{Z}_q[x]/(x^N + 1)$ can be equivalently computed in the quotient ring $[\mathbb{Z}_q[y]/(y^n + 1)][x]/(x^d - y)$. Thus, we have the following relation in the mapped ring:

$$\sum_{i=0}^{d-1} h^{(i)}(y) x^i = \left(\sum_{u=0}^{d-1} f^{(u)}(y) x^u \right) \left(\sum_{v=0}^{d-1} g^{(v)}(y) x^v \right).$$

Note that $\mathcal{H}_i = (h_i, h_{i+d}, h_{i+2d}, \dots, h_{i+(n-1)d})$ coincides exactly with the coefficient vector of $h^{(i)}(y)$. By equating the coefficients of x^i on both sides and reducing any exponents $\geq d$ via the relation $x^d = y$, the coefficient $h^{(i)}(y)$ can be explicitly written as

$$h^{(i)}(y) = \sum_{s=0}^i f^{(i-s)}(y) \cdot g^{(s)}(y) + \sum_{s=i+1}^{d-1} \mathcal{S}(f^{(d+i-s)}(y)) \cdot g^{(s)}(y), \quad (4)$$

where $\mathcal{S}(\cdot)$ denotes the cyclic shift operator in $\mathbb{Z}_q[y]/(y^n + 1)$ induced by multiplication by y , specifically,

$$\mathcal{S}(\nu(y)) = y \cdot \nu(y) = -\nu_{n-1} + \sum_{j=1}^{n-1} \nu_{j-1} y^j,$$

with $\nu(y) = \sum_{j=0}^{n-1} \nu_j y^j \in \mathbb{Z}_q[y]/(y^n + 1)$.

Since $f, g \in \mathcal{R}_q = \mathbb{Z}_q[x]/(x^N + 1)$ have coefficients drawn independently from distributions ψ_f and ψ_g , respectively, it can be observed that $h^{(i)}(y)$ in (4) is a sum of d independent polynomials in $\mathbb{Z}_q[y]/(y^n + 1)$:

- the first $i + 1$ polynomials correspond to the product of two polynomials $c = a \cdot b \in \mathbb{Z}_q[y]/(y^n + 1)$, where a and b have coefficients drawn independently from ψ_f and ψ_g , respectively;
- the remaining $d - i - 1$ polynomials correspond to a similar product, with a replaced by its cyclic shift $\mathcal{S}(a)$.

Since ψ_f is symmetric, the distribution of $f^{(d+i-s)}(y)$ is invariant under the cyclic shift operator $\mathcal{S}(\cdot)$ in (4). Hence, each $\mathcal{H}_i$ is identically distributed and can be expressed as a sum of d independent random variables following the distribution $\text{Prod}_{\psi_f, \psi_g}$, i.e.,

$$\mathcal{H}_i = \sum_{j=0}^{d-1} \mathcal{J}_j, \quad \text{with} \quad \{\mathcal{J}_j\}_{j=0}^{d-1} \stackrel{\text{i.i.d.}}{\sim} \text{Prod}_{\psi_f, \psi_g}.$$

□

Now consider the products $\mathbf{e}^T \mathbf{r}$ and $\mathbf{s}^T (\mathbf{e}_1 + \boldsymbol{\epsilon}_u)$ in the noise polynomial Err . As specified in Figure 2 and Theorem 3, these polynomial vectors are mutually independent, and the components within each vector are independent and identically distributed. Moreover, the distributions of $\mathbf{e}$ and $\mathbf{s}$ are symmetric. Consequently, both products can be regarded as independent sums of l identically distributed polynomial multiplications that follow the structure characterized in Lemma 4. This observation immediately leads to the following corollary, which characterizes the joint distribution of the n coefficients in each block $(\text{Err})_i$. For brevity, its proof is omitted.

Corollary 1 *Let $\text{Err} = \mathbf{e}^T \mathbf{r} - \mathbf{s}^T (\mathbf{e}_1 + \boldsymbol{\epsilon}_u) + e_2 + \epsilon_v$ be defined as in Theorem 3. Define the distribution φ_d as follows: sample a random $x \xleftarrow{\$} \mathbb{Z}$ and output $(x - \text{Decompress}_{2^d, q}(\text{Compress}_{q, 2^d}(x))) \bmod \pm q$. Then, for each $i \in \{0, 1, \dots, d-1\}$ with $d = N/n$, the n -dimensional coefficient vector $(\text{Err})_i$ follows an identical distribution, which can be expressed as a sum of independent random variables:*

$$(\text{Err})_i = \sum_{j=0}^{ld-1} \mathcal{X}_j + \sum_{j=0}^{ld-1} \mathcal{Y}_j + (e_2)_i + (\epsilon_v)_i,$$

where $\{\mathcal{X}_j\}_{j=0}^{ld-1} \stackrel{\text{i.i.d.}}{\sim} \text{Prod}_{\mathcal{B}_{\eta_1}, \mathcal{B}_{\eta_1}}$ and $\{\mathcal{Y}_j\}_{j=0}^{ld-1} \stackrel{\text{i.i.d.}}{\sim} \text{Prod}_{\mathcal{B}_{\eta_1}, \mathcal{B}_{\eta_2} + \varphi_{d_u}}$. Moreover, $(e_2)_i$ and $(\epsilon_v)_i$ are n -dimensional coefficient vectors whose components are independently drawn from $\mathcal{B}_{\eta_2}$ and φ_{d_v} , respectively.

The above analysis clarifies the rationale behind the encoding polynomial coefficient embedding method illustrated in Figure 2. To apply Corollary 1 for estimating the DFR of BW-KEM, each μ_0 -bit message should be encoded into n coefficients spaced every $\frac{N}{n}$ entries. Since each block $(\text{Err})_i$ follows the same distribution, a union bound on the DFR δ can be derived as

$$\begin{aligned} \delta &= 1 - \Pr [\|(\text{Err})_i\|_{q,2} < r_0, \forall i] \\ &\leq \sum_{i=0}^{\frac{N}{n}-1} \Pr [\|(\text{Err})_i\|_{q,2} \geq r_0] = \frac{N}{n} \Pr [\|(\text{Err})_0\|_{q,2} \geq r_0]. \end{aligned}$$

When n is relatively small, $\Pr[\|(\text{Err})_0\|_{q,2} \geq r_0]$ can be evaluated directly by computing the corresponding n -dimensional joint distribution. However, as n increases, the computational complexity grows exponentially, making such direct evaluation intractable for $n \geq 8$. To address this challenge, we introduce in the next section a distribution approximation method that enables efficient estimation of the decoding failure rate for larger values of n .

5.2 Asymptotic Approximation of the Noise Distribution for DFR Estimation

The accumulated noise Err that needs to be corrected during decryption consists of two main components: an approximately Gaussian part arising from $\mathbf{e}^T \mathbf{r} - \mathbf{s}^T(\mathbf{e}_1 + \boldsymbol{\epsilon}_{\mathbf{u}}) + e_2$, and a nearly uniform part resulting from ϵ_v . As n increases, the complexity of directly computing the n -dimensional joint distribution of the product terms $\mathbf{e}^T \mathbf{r}$ and $\mathbf{s}^T(\mathbf{e}_1 + \boldsymbol{\epsilon}_{\mathbf{u}})$ becomes computationally prohibitive. To overcome this limitation, we introduce an approximation methodology for characterizing the joint distribution of the coefficients.

Note that both product terms can be represented as independent sums of l polynomial multiplications with small-norm coefficients. We establish the following theorem to show that, as lN tends to infinity, the joint distribution of the coefficients converges to a multivariate normal distribution. Furthermore, our analytical framework can be readily generalized to other polynomial rings frequently employed in cryptographic constructions, such as $\mathbb{Z}_q[x]/(x^N - x - 1)$ [4] and $\mathbb{Z}_q[x]/(x^N - x^{N/2} + 1)$ [17].

Theorem 6 *Let $\mathbf{f} = (\mathbf{f}[0], \mathbf{f}[1], \dots, \mathbf{f}[l-1])^T$ and $\mathbf{g} = (\mathbf{g}[0], \mathbf{g}[1], \dots, \mathbf{g}[l-1])^T$ be two independent polynomial vectors, where $\mathbf{f}[j], \mathbf{g}[j]$ are polynomials in $\mathbb{R}[x]/(x^N + 1)$ with coefficients drawn independently from $\mathcal{N}(0, \sigma_f^2)$ and $\mathcal{N}(0, \sigma_g^2)$, respectively, for $j = 0, 1, \dots, l-1$. Let $h(x) = \mathbf{f}^T \mathbf{g} = \sum_{j=0}^{l-1} \mathbf{f}[j] \mathbf{g}[j] \in \mathbb{R}[x]/(x^N + 1)$. For any fixed integers $0 \leq c_1 < c_2 < N$, as lN tends to infinity, the distribution of $\left(\frac{h_{c_1}}{\sigma_f \sigma_g \sqrt{lN}}, \frac{h_{c_2}}{\sigma_f \sigma_g \sqrt{lN}}\right)$ converges in distribution to the bivariate standard normal distribution $\mathcal{N}(\mathbf{0}, \mathbf{I}_2)$.*

Proof. Let $\phi_{c_1, c_2}(t_1, t_2)$ denote the characteristic function of the random vector $\left(\frac{h_{c_1}}{\sigma_f \sigma_g \sqrt{lN}}, \frac{h_{c_2}}{\sigma_f \sigma_g \sqrt{lN}}\right)$, i.e.,

$$\phi_{c_1, c_2}(t_1, t_2) = \mathbb{E} \left[\exp \left(i \left(t_1 \frac{h_{c_1}}{\sigma_f \sigma_g \sqrt{lN}} + t_2 \frac{h_{c_2}}{\sigma_f \sigma_g \sqrt{lN}} \right) \right) \right].$$

Denote $\mathbf{f}_j = (\mathbf{f}[j]_0, \mathbf{f}[j]_1, \dots, \mathbf{f}[j]_{N-1})^T$ and $\mathbf{g}_j = (\mathbf{g}[j]_0, \mathbf{g}[j]_1, \dots, \mathbf{g}[j]_{N-1})^T$. Let $\mathbf{J}_r$ be a square matrix of order r whose entries are all zeros except those on the anti-diagonal, which are all ones. Define

$\mathbf{A}_{c_i} = \text{diag}\{\mathbf{J}_{c_i+1}, -\mathbf{J}_{N-c_i-1}\}$ for $0 \leq c_i < N$, i.e.,

$$\mathbf{A}_{c_i} = \begin{pmatrix} \mathbf{J}_{c_i+1} & \mathbf{O} \\ \mathbf{O} & -\mathbf{J}_{N-c_i-1} \end{pmatrix}_{N \times N}, \mathbf{J}_i = \begin{pmatrix} & & 1 \\ & \ddots & \\ 1 & & \end{pmatrix}_{i \times i}.$$

It is straightforward to verify that $\mathbf{A}_{c_i}$ satisfies the equation $h_{c_i} = \sum_{j=0}^{l-1} \mathbf{f}_j^T \mathbf{A}_{c_i} \mathbf{g}_j$. Then we obtain

$$t_1 \frac{h_{c_1}}{\sigma_f \sigma_g \sqrt{lN}} + t_2 \frac{h_{c_2}}{\sigma_f \sigma_g \sqrt{lN}} = \sum_{j=0}^{l-1} \frac{\mathbf{f}_j^T}{\sigma_f} \left(\frac{t_1}{\sqrt{lN}} \mathbf{A}_{c_1} + \frac{t_2}{\sqrt{lN}} \mathbf{A}_{c_2} \right) \frac{\mathbf{g}_j}{\sigma_g}.$$

As $\left(\frac{t_1}{\sqrt{lN}} \mathbf{A}_{c_1} + \frac{t_2}{\sqrt{lN}} \mathbf{A}_{c_2} \right)$ is symmetric, it can be orthogonally diagonalized as $\mathbf{P}^T \mathbf{\Lambda} \mathbf{P}$, where $\mathbf{P}$ is orthogonal and $\mathbf{\Lambda}$ is diagonal. Hence, the characteristic function satisfies

$$\phi_{c_1, c_2}(t_1, t_2) = \mathbb{E} \left[\exp \left(i \sum_{j=0}^{l-1} \frac{(\mathbf{P} \mathbf{f}_j)^T}{\sigma_f} \mathbf{\Lambda} \frac{(\mathbf{P} \mathbf{g}_j)}{\sigma_g} \right) \right] = \prod_{j=0}^{l-1} \mathbb{E} \left[\exp \left(i \frac{(\mathbf{P} \mathbf{f}_j)^T}{\sigma_f} \mathbf{\Lambda} \frac{(\mathbf{P} \mathbf{g}_j)}{\sigma_g} \right) \right].$$

Since $\mathbf{P}$ is orthogonal, it preserves the normal distribution. Thus, $\frac{(\mathbf{P} \mathbf{f}_j)^T}{\sigma_f} \mathbf{\Lambda} \frac{(\mathbf{P} \mathbf{g}_j)}{\sigma_g}$ equals the sum of N scaled products of two independent one-dimensional Gaussian variables.

Suppose $\{\lambda_i\}_{k=0}^{N-1}$ are the eigenvalues of $\left(\frac{t_1}{\sqrt{lN}} \mathbf{A}_{c_1} + \frac{t_2}{\sqrt{lN}} \mathbf{A}_{c_2} \right)$, and let ϕ is the characteristic function of the product of two independent one-dimensional standard Gaussian variables. Since $\phi(t) = (1 + t^2)^{-\frac{1}{2}}$ [21], we obtain

$$\phi_{c_1, c_2}(t_1, t_2) = \prod_{j=0}^{l-1} \prod_{k=0}^{N-1} \phi(\lambda_k) = \prod_{j=0}^{l-1} \prod_{k=0}^{N-1} (1 + \lambda_k^2)^{-\frac{1}{2}}.$$

Then, for fixed t_1, t_2 , we have

$$\ln(\phi_{c_1, c_2}(t_1, t_2)) = -\frac{1}{2} \sum_{j=0}^{l-1} \sum_{k=0}^{N-1} \ln(1 + \lambda_k^2) = -\frac{l}{2} \sum_{k=0}^{N-1} (\lambda_k^2 + r_k), \quad (5)$$

where r_k denotes the Lagrange remainder term, satisfying the bound $|r_k| \leq \frac{\lambda_k^4}{2}$. By the Gershgorin Circle Theorem, we obtain $\lambda_k^2 \leq \left(\frac{2|t_1|}{\sqrt{lN}} + \frac{2|t_2|}{\sqrt{lN}} \right)^2 \leq \frac{4(|t_1| + |t_2|)^2}{lN}$. Hence, the summation of the remainder terms satisfies

$$\left| l \sum_{k=0}^{N-1} r_k \right| \leq l \sum_{k=0}^{N-1} |r_k| \leq l \sum_{k=0}^{N-1} \frac{\lambda_k^4}{2} \leq \frac{8(|t_1| + |t_2|)^4}{lN}. \quad (6)$$

For a square matrix $\mathbf{A}$, let $\text{tr}(\mathbf{A})$ denote the sum of the elements on its main diagonal. From the definition of $\mathbf{A}_{c_i}$, it follows that $\text{tr}(\mathbf{A}_{c_i}^2) = N$ and $\text{tr}(\mathbf{A}_{c_1}\mathbf{A}_{c_2}) = 0$. Thus, for $\sum_{k=0}^{N-1} \lambda_k^2$, we further observe that

$$\begin{aligned} \sum_{k=0}^{N-1} \lambda_k^2 &= \text{tr} \left(\frac{t_1}{\sqrt{lN}} \mathbf{A}_{c_1} + \frac{t_2}{\sqrt{lN}} \mathbf{A}_{c_2} \right)^2 \\ &= \frac{t_1^2}{lN} \text{tr}(\mathbf{A}_{c_1}^2) + \frac{t_2^2}{lN} \text{tr}(\mathbf{A}_{c_2}^2) + \frac{2t_1t_2}{lN} \text{tr}(\mathbf{A}_{c_1}\mathbf{A}_{c_2}) \\ &= (t_1^2 + t_2^2)/l. \end{aligned}$$

Combining this equation with (5) and (6), we conclude that as lN tends to infinity, $\phi_{c_1, c_2}(t_1, t_2)$ converges pointwise to $\exp(-(t_1^2 + t_2^2)/2)$, which is the characteristic function of the bivariate normal distribution $\mathcal{N}(\mathbf{0}, \mathbf{I}_2)$. By Lévy's continuity theorem, we deduce that the random vector $\left(\frac{h_{c_1}}{\sigma_f \sigma_g \sqrt{lN}}, \frac{h_{c_2}}{\sigma_f \sigma_g \sqrt{lN}} \right)$ converges in distribution to $\mathcal{N}(\mathbf{0}, \mathbf{I}_2)$. $\square$

Let σ_i ($i = 1, 2$) denote the standard deviation of the centered binomial distribution B_{η_i} , and let σ_{ϵ_u} denote the standard deviation of the distribution ϵ_u . According to Theorem 6, as $lN \rightarrow \infty$, the n -dimensional joint distributions of coefficients of $\mathbf{e}^T \mathbf{r}$ and $\mathbf{s}^T(\mathbf{e}_1 + \epsilon_u)$ converge in distribution to the multivariate normal distributions $\mathcal{N}(\mathbf{0}, lN\sigma_1^4 \mathbf{I}_n)$ and $\mathcal{N}(\mathbf{0}, lN\sigma_1^2(\sigma_2^2 + \sigma_{\epsilon_u}^2) \mathbf{I}_n)$, respectively. Let $\sigma^2 = lN\sigma_1^2(\sigma_1^2 + \sigma_2^2 + \sigma_{\epsilon_u}^2)$. Since the two product terms are independent, we approximate the joint distribution of $(\mathbf{e}^T \mathbf{r} - \mathbf{s}^T(\mathbf{e}_1 + \epsilon_u))_i$ in $(\text{Err})_i$ by $\mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_n)$ when evaluating the DFR under concrete parameter settings. Moreover, we conduct extensive numerical experiments to further confirm the reasonableness of this approximation for our proposed parameter sets, where lN is at least 512.

6 Concrete Security and Parameter Selection

6.1 Core-SVP Hardness

Since the best known attacks against the MLWE problem do not exploit the underlying lattice structure, the hardness of MLWE can be analyzed in terms of the standard LWE problem. For concrete hardness estimation in most practical lattice-based cryptosystems, the primary attacks considered are two BKZ-based [7, 19] strategies, namely the primal and the dual attack.

Primal Attack The primal attack aims to solve the unique-SVP by constructing an instance from the LWE problem. Let $m = (l + 1)n$ denote the number of LWE samples available to the adversary. Given the matrix LWE instance $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{m \times ln} \times \mathbb{Z}_q^m$, the primal attack constructs a lattice $\Lambda = \{\mathbf{x} \in \mathbb{Z}_q^{m+kn+1} \mid (\mathbf{A} \mid \mathbf{I}_m \mid -\mathbf{b})\mathbf{x} = \mathbf{0} \pmod{q}\}$ and then applies the BKZ lattice reduction algorithm to find the unique-SVP solution $\mathbf{v} = (\mathbf{s}, \mathbf{e}, 1)$. We refer the reader to [2] for a detailed analysis of the primal attack.

Dual Attack The dual attack tries to distinguish the matrix LWE instance $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}) \in \mathbb{Z}_q^{m \times ln} \times \mathbb{Z}_q^m$ from a uniformly random sample by identifying a short vector $(\mathbf{v}, \mathbf{w})$ in the dual lattice $\Lambda' = \{(\mathbf{x}, \mathbf{y}) \in \mathbb{Z}_q^m \times \mathbb{Z}_q^{ln} \mid \mathbf{A}^T \mathbf{x} = \mathbf{y} \pmod{q}\}$. Suppose such a vector $(\mathbf{v}, \mathbf{w})$ is found. One then computes $z = \mathbf{v}^T \mathbf{b} = \mathbf{v}^T \mathbf{A} \mathbf{s} + \mathbf{v}^T \mathbf{e} = \mathbf{w}^T \mathbf{s} + \mathbf{v}^T \mathbf{e}$, which is small and approximately Gaussian distributed if $(\mathbf{A}, \mathbf{b})$ corresponds to an LWE sample; otherwise, z is uniformly distributed modulo q . We refer the reader to [1, 6] for a detailed analysis of the dual attack.

Following the simple and conservative methodology of the core-SVP hardness [2], the best known cost of solving the b -dimensional SVP is estimated to be $2^{0.292b}$ in the classical setting and $2^{0.265b}$ in the quantum setting. These cost models are commonly adopted for conservative security estimates of lattice-based schemes. We estimate the classical and quantum core-SVP hardness security of BW-KEM using the Python script from [2, 6, 20]. The concrete results are summarized in Table 2.

6.2 Parameter Sets

We first consider the parameters of the scalable nested lattice code $\mathcal{C}_{n,\tau}(\lambda)$ used in our lattice coding framework. According to the construction of BW-KEM, the length of the shared key is given by $\frac{N}{n} \mu_0$, where $\mu_0 = \tau n - \frac{n}{4}(k-1)$. The corresponding parameters of $\mathcal{C}_{n,\tau}(\lambda)$ are listed in Table 1.

Table 1. Parameters for $\mathcal{C}_{n,\tau}(\lambda)$.

SecLev	n	τ	μ_0	Coding Lattice	Shaping Lattice
c128	8	2	4	$\text{BW}_8(\frac{q}{4}), \text{BW}_8(\frac{\hat{q}}{4})$	$q \cdot \mathbb{G}^4, \hat{q} \cdot \mathbb{G}^4$
Others	32	2	32	$\text{BW}_{32}(\frac{q}{4}), \text{BW}_{32}(\frac{\hat{q}}{4})$	$q \cdot \mathbb{G}^{16}, \hat{q} \cdot \mathbb{G}^{16}$

For security preservation, we provide four recommended parameter sets for BW-KEM, namely BW-KEM-c128, BW-KEM-c256, BW-KEM-c384, and BW-KEM-c512, corresponding to the required security levels. In addition, for each security level, we offer a ‘-s’ variant as an alternative configuration, which increase the compression parameters in order to reduce the decryption failure probability.

Table 2 reports the main parameters, the security parameter κ , the public key size $|\mathbf{pk}|$, ciphertext size $|\mathbf{ct}|$, total bandwidth $|\mathbf{pk}| + |\mathbf{ct}|$, and shared key size $|\mathbf{K}|$ in bytes. Here l denotes the module rank, while κ denotes the bit length of seeds and key strings used by the KEM. The columns labeled “(sec.c, sec.q)” indicate the estimated classical and quantum security levels, respectively. The quantities δ denotes the decryption-failure probabilities of BW-KEM.

Table 2. Parameter sets of BW-KEM.

Scheme	Lattice	N	q	l	λ	$(\eta_s, \eta_e, \eta_{ct})$	d_u	d_v	(sec.c, sec.q)	δ	$ pk $	$ ct $	$ pk + ct $	$ K $
Recommended parameter sets														
BW-KEM-c128	BW ₈	256	3329	2	128	(6, 6, 5)	10	4	(131, 119)	$2^{-138.14}$	784	768	1552	16
BW-KEM-c256	BW ₃₂	256	3329	4	256	(3, 3, 3)	10	5	(267, 242)	$2^{-209.94}$	1568	1440	3008	32
BW-KEM-c384	BW ₃₂	512	3329	3	512	(2, 2, 2)	10	4	(403, 365)	$2^{-216.26}$	2368	2176	4544	64
BW-KEM-c512	BW ₃₂	512	3329	4	512	(2, 2, 2)	10	6	(556, 504)	$2^{-232.71}$	3136	2944	6080	64
Strengthened parameter sets														
BW-KEM-c128-s	BW ₈	256	3329	2	128	(6, 6, 5)	11	5	(131, 119)	$2^{-188.01}$	784	928	1712	16
BW-KEM-c256-s	BW ₃₂	256	3329	4	256	(3, 3, 3)	11	6	(267, 242)	$2^{-277.92}$	1568	1600	3168	32
BW-KEM-c384-s	BW ₃₂	512	3329	3	512	(2, 2, 2)	11	5	(403, 365)	$2^{-370.79}$	2368	2432	4800	64
BW-KEM-c512-s	BW ₃₂	512	3329	4	512	(2, 2, 2)	11	7	(556, 504)	$2^{-308.56}$	3136	3264	6400	64

7 Implementation and Benchmark

7.1 Implementation Overview

This section describes the software implementation of BW-KEM and reports performance benchmarks. All measurements follow the NICCS x86 implementation self-evaluation methodology: every algorithm instance, security level, and implementation version is covered, and each reported quantity is the median over at least 1,000 executions.

The submission provides one platform-independent *reference* implementation written in portable ISO C99, and one *optimized* implementation targeting mainstream 64-bit x86 processors with the AVX2 instruction-set extension. Following the NICCS self-evaluation guideline, the optimized implementation is further split into a *performance-optimized* variant and a *resource-optimized* variant. All three versions expose the identical ICCS KEM programming interface (`kem_keygen`, `kem_enc`, `kem_dec`) and produce byte-identical known-answer test (KAT) vectors, so the optimized variants are drop-in replacements for the reference code. BW-KEM provides three parameter sets, BW-KEM-c128, BW-KEM-c256, and BW-KEM-c512, targeting the 128-, 256-, and 512-bit classical security levels, respectively. All parameter sets share the modulus $q = 3329$ and the polynomial arithmetic of ML-KEM, which allows the implementation to reuse the bulk of existing ML-KEM-style software modules and hardware-acceleration primitives such as number-theoretic transform (NTT) units. The only algorithmic addition relative to ML-KEM is the scalable nested Barnes-Wall lattice codec (Section 3), which replaces direct modulation in the message-encoding path.

In accordance with the NICCS submission requirements, all hashing and extendable-output functionality is instantiated with the SM3-based auxiliary functions supplied by ICCS rather than the SHA-3/SHAKE primitives used by ML-KEM; the hash functions, the pseudorandom function (PRF), and the extendable-output function (XOF) used in matrix expansion and noise sampling are all derived from SM3. The reference implementation uses a single-lane scalar

SM3 core, whereas the performance-optimized implementation additionally provides an 8-way parallel SM3 engine on 256-bit AVX2 registers together with vectorized PRF/XOF wrappers. The performance-optimized implementation vectorizes the dominant kernels in AVX2: the NTT, inverse NTT, and base multiplication (with register-resident Montgomery butterflies and lazy reduction; C128 and C256 use the standard 7-layer ML-KEM-style transform on 256-coefficient polynomials, while C512 uses a dedicated incomplete NTT for $N = 512$ with degree-4 base multiplication), the Barnes-Wall message codec, the batched 4-way centered-binomial noise sampling, and the matrix generation with vectorized rejection sampling. The Fujisaki-Okamoto re-encryption, the constant-time ciphertext comparison, and the conditional move in decapsulation are deliberately kept scalar and branch-free in all versions to preserve their constant-time properties; they are not on the performance-critical path.

All experiments were conducted on a single core of an Intel[®] Xeon[®] Platinum 8575C processor with 16 GB of main memory, running Ubuntu 22.04.5 LTS (Linux kernel 5.15), with all code compiled by GCC 12.3.0. To ensure comparability, the compilation options follow the NICCS x86 self-evaluation guideline: the reference version uses `-std=c99 -Wpedantic -Wall -Wextra -O2`; the performance-optimized version uses `-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer`; and the resource-optimized version uses `-Os -march=x86-64 -mavx2 -flto -fomit-frame-pointer`. Each operation was executed at least 1,000 times and the median CPU-cycle count is reported. Functional correctness was verified for every instance and every version by checking the consistency of the known-answer test vectors; all cases passed.

7.2 Performance and Benchmark Results

Table 3 reports the median cycle counts and the corresponding throughput (operations per second) for key generation, encapsulation, and decapsulation, across all three parameter sets and all three implementation versions. The performance-optimized AVX2 implementation achieves roughly a 4.8–7.4 $\times$ speedup over the reference implementation at the full KEM level, with the largest gains at the higher security levels where the vectorized NTT and base multiplication dominate; the resource-optimized variant trades part of this speedup for a smaller code footprint.

The public-key, secret-key, ciphertext, and shared-secret sizes are determined by the algorithm specification and are therefore identical across all three implementation versions; they are listed in Table 4. Table 5 reports the static memory footprint (text + data + bss of the self-evaluation binary, as reported by the `size` utility) and the peak resident-set size observed during execution (via `getrusage`). The performance-optimized version has the largest static footprint because of the additional AVX2 assembly kernels and the 8-way SM3 engine, whereas the resource-optimized version is the most compact while retaining most of the AVX2 speedup.

Table 3. Performance of BW-KEM (median cycles and throughput in operations per second). “ref” is the reference implementation, “perf” the performance-optimized AVX2 version, and “res” the resource-optimized AVX2 version.

Instance	Op.	Cycles (median)			Throughput (ops/s)		
		ref	perf	res	ref	perf	res
BW-KEM-c128	KeyGen	235,934	48,906	144,366	11,389	55,738	17,997
	Encaps	265,958	64,470	173,824	10,373	42,924	17,915
	Decaps	308,792	81,178	133,130	8,932	33,798	17,762
BW-KEM-c256	KeyGen	583,280	87,844	294,660	4,765	30,292	9,435
	Encaps	619,266	129,388	304,750	4,445	21,229	8,868
	Decaps	718,464	170,150	348,960	3,681	15,766	7,795
BW-KEM-c512	KeyGen	1,979,672	267,224	1,009,530	1,399	10,308	2,726
	Encaps	1,971,860	298,122	1,038,908	1,375	9,286	2,646
	Decaps	2,296,972	451,896	1,172,090	1,213	6,220	2,277

Table 4. Transfer and storage sizes of BW-KEM (bytes). Identical across the reference and optimized implementations.

Instance	Public key	Secret key	Ciphertext	Shared secret
BW-KEM-c128	784	1,585	768	16
BW-KEM-c256	1,568	3,169	1,440	32
BW-KEM-c512	3,136	6,337	2,944	64

Overall, the benchmarks confirm the design goals of BW-KEM. Because BW-KEM preserves the modulus and polynomial dimensions of ML-KEM, the AVX2-optimized arithmetic kernels deliver substantial speedups (up to $7.4\times$ at the full KEM level) while the additional Barnes-Wall codec contributes only a small overhead; the constant-time message codec relies solely on integer arithmetic and bitwise logic, keeping the FO transform and the comparison logic branch-free across all versions; and the availability of both a performance-optimized and a resource-optimized AVX2 build allows the same algorithm to serve both high-throughput servers and resource-constrained software platforms without modifying the algorithm interface or the test vectors.

7.3 ARM Cortex-M4 Implementation and Benchmarking

We additionally provide optimized ARM Cortex-M4 implementations of BW-KEM and benchmark them on the `stm32f4discovery` platform, using an STM32F407 microcontroller with a Cortex-M4F core. The firmware is cross-compiled with `arm-none-eabi-gcc` 13.2.1 using the Cortex-M4 build configura-

Table 5. Resource consumption of BW-KEM (bytes): static memory (text + data + bss) and peak resident-set size (RSS).

Instance	Static memory			Peak RSS		
	ref	perf	res	ref	perf	res
BW-KEM-c128	40,395	69,308	36,656	2,732,032	2,899,968	2,891,776
BW-KEM-c256	48,190	71,700	40,735	2,957,312	2,928,640	2,916,352
BW-KEM-c512	48,886	95,329	68,992	2,920,448	2,846,720	2,945,024

tion -mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpu=fpv4-sp-d16. The board tests were executed through the serial interface after successful ST-Link flashing, and all functional tests passed.

The ARM submission contains three variants for each of BW-KEM-c128, BW-KEM-c256, and BW-KEM-c512. The **ref** variant is the portable reference implementation for correctness and comparison; the **m4fspeed** variant is optimized for speed on Cortex-M4; and the **m4fstack** variant is optimized for stack usage. All three variants expose the same KEM API, namely **crypto_kem_keypair**, **crypto_kem_enc**, and **crypto_kem_dec**. The optimized ARM variants produce KAT files whose SHA-256 digests match the corresponding reference KAT files. For benchmarking, the **m4fspeed** speed test and the **m4fstack** stack-usage test each use 10,000 repetitions for every parameter set and KEM operation, and all clock cycle counts and stack values reported below are averages over these 10,000 measurements.

Table 6 reports the ARM Cortex-M4 clock cycle counts for key generation, encapsulation, and decapsulation. Compared with the ARM reference implementation, the **m4fspeed** implementation reduces the total KEM cycle count, computed as the sum of key generation, encapsulation, and decapsulation, by $2.22\times$, $2.44\times$, and $3.85\times$ for BW-KEM-c128, BW-KEM-c256, and BW-KEM-c512, respectively. The larger gain for BW-KEM-c512 reflects the increasing share of polynomial arithmetic in the overall running time, where the Cortex-M4-specific kernels are most effective.

Table 6. Speed of BW-KEM on ARM Cortex-M4 (Unit: clock cycle).

Parameter Set	Scheme	KeyGen	Encaps	Decaps
BW-KEM-c128	ref	1,004,381	1,187,080	1,449,064
	m4fspeed	485,572	537,032	616,653
BW-KEM-c256	ref	2,490,610	2,700,371	3,276,898
	m4fspeed	1,086,200	1,112,666	1,267,009
BW-KEM-c512	ref	8,033,328	8,501,500	9,992,174
	m4fspeed	2,149,122	2,214,974	2,532,809

Table 7 reports the measured stack usage on ARM Cortex-M4. Using the maximum over the three KEM operations as the per-instance stack requirement, the `m4fstack` implementation reduces stack consumption by 56.1%, 72.1%, and 74.0% for BW-KEM-c128, BW-KEM-c256, and BW-KEM-c512, respectively, relative to the ARM reference implementation.

Table 7. Stack usage of BW-KEM on ARM Cortex-M4 (Unit: byte).

Parameter Set	Scheme	KeyGen	Encaps	Decaps
BW-KEM-c128	ref	7,788	10,388	11,147
	m4fstack	4,345	4,905	4,897
BW-KEM-c256	ref	16,732	20,193	21,609
	m4fstack	5,461	6,053	6,037
BW-KEM-c512	ref	31,521	38,799	41,687
	m4fstack	9,743	10,911	10,855

References

1. Albrecht, M.R.: On dual lattice attacks against small-secret LWE and parameter choices in HELib and SEAL. In: Coron, J.S., Nielsen, J.B. (eds.) *Advances in Cryptology – EUROCRYPT 2017, Part II*. Lecture Notes in Computer Science, vol. 10211, pp. 103–129. Springer, Cham, Switzerland, Paris, France (Apr 30 – May 4, 2017). https://doi.org/10.1007/978-3-319-56614-6_4
2. Alkim, E., Ducas, L., Pöppelmann, T., Schwabe, P.: Post-quantum key exchange - A new hope. In: Holz, T., Savage, S. (eds.) *USENIX Security 2016: 25th USENIX Security Symposium*. pp. 327–343. USENIX Association, Austin, TX, USA (Aug 10–12, 2016), <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/alkim>
3. Bellare, M., Rogaway, P.: Random oracles are practical: A paradigm for designing efficient protocols. In: Denning, D.E., Pyle, R., Ganesan, R., Sandhu, R.S., Ashby, V. (eds.) *ACM CCS 93: 1st Conference on Computer and Communications Security*. pp. 62–73. ACM Press, Fairfax, Virginia, USA (Nov 3–5, 1993). <https://doi.org/10.1145/168588.168596>
4. Bernstein, D.J., Brumley, B.B., Chen, M.S., Chuengsatiansup, C., Lange, T., Marotzke, A., Peng, B.Y., Tuveri, N., van Vredendaal, C., Yang, B.Y.: NTRU Prime. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>
5. Boneh, D., Dagdelen, Ö., Fischlin, M., Lehmann, A., Schaffner, C., Zhandry, M.: Random oracles in a quantum world. In: Lee, D.H., Wang, X. (eds.) *Advances in Cryptology – ASIACRYPT 2011*. Lecture Notes in Computer Science, vol. 7073, pp. 41–69. Springer Berlin Heidelberg, Germany, Seoul, South Korea (Dec 4–8, 2011). https://doi.org/10.1007/978-3-642-25385-0_3
6. Carrier, K., Meyer-Hilfiger, C., Shen, Y., Tillich, J.P.: Assessing the impact of a variant of MATZOV’s dual attack on kyber. In: Kalai, Y.T., Kamara, S.F. (eds.) *Advances in Cryptology – CRYPTO 2025, Part I*. Lecture Notes in Computer Science, vol. 16000, pp. 444–476. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2025). https://doi.org/10.1007/978-3-032-01855-7_15
7. Chen, Y., Nguyen, P.Q.: BKZ 2.0: Better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) *Advances in Cryptology – ASIACRYPT 2011*. Lecture Notes in Computer Science, vol. 7073, pp. 1–20. Springer Berlin Heidelberg, Germany, Seoul, South Korea (Dec 4–8, 2011). https://doi.org/10.1007/978-3-642-25385-0_1
8. Conway, J.H., Sloane, N.J.A.: *Sphere packings, lattices and groups*. Springer New York, NY, 3rd edn. (1999)
9. Corlay, V., Boutros, J.J., Ciblât, P., Brunel, L.: On the decoding of barnes-wall lattices. In: *2020 IEEE International Symposium on Information Theory (ISIT)*. pp. 519–524 (2020). <https://doi.org/10.1109/ISIT44484.2020.9173976>
10. Duman, J., Hövelmanns, K., Kiltz, E., Lyubashevsky, V., Seiler, G.: Faster lattice-based KEMs via a generic Fujisaki-Okamoto transform using prefix hashing. In: Vigna, G., Shi, E. (eds.) *ACM CCS 2021: 28th Conference on Computer and Communications Security*. pp. 2722–2737. ACM Press, Virtual Event, Republic of Korea (Nov 15–19, 2021). <https://doi.org/10.1145/3460120.3484819>
11. Forney, G.D.: Coset codes. ii. binary lattices and related codes. *IEEE Transactions on Information Theory* **34**(5), 1152–1187 (Sep 1988). <https://doi.org/10.1109/18.21246>

12. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: Wiener, M.J. (ed.) *Advances in Cryptology – CRYPTO’99*. Lecture Notes in Computer Science, vol. 1666, pp. 537–554. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 15–19, 1999). https://doi.org/10.1007/3-540-48405-1_34
13. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the Fujisaki-Okamoto transformation. In: Kalai, Y., Reyzin, L. (eds.) *TCC 2017: 15th Theory of Cryptography Conference, Part I*. Lecture Notes in Computer Science, vol. 10677, pp. 341–371. Springer, Cham, Switzerland, Baltimore, MD, USA (Nov 12–15, 2017). https://doi.org/10.1007/978-3-319-70500-2_12
14. Langlois, A., Stehlé, D.: Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography* **75**(3), 565–599 (2015). <https://doi.org/10.1007/s10623-014-9938-4>
15. Li, S., Zou, H., Shen, S., Dong, Y., Zhao, Y.: BW-KEM: Robust and versatile MLWE-based KEM with Barnes-Wall lattices. In: Bai, S., Persichetti, E. (eds.) *Public-Key Cryptography – PKC 2026*. Lecture Notes in Computer Science, vol. 16551, pp. 70–102. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-26731-3_3
16. Liu, Y., Zhang, Y., Lu, X., Cheng, Y., Yin, Y.: DAWN: Smaller and faster NTRU encryption via double encoding. In: Hanaoka, G., Yang, B.Y. (eds.) *Advances in Cryptology – ASIACRYPT 2025, Part III*. Lecture Notes in Computer Science, vol. 16247, pp. 396–427. Springer, Singapore, Singapore, Melbourne, VIC, Australia (Dec 8–12, 2025). https://doi.org/10.1007/978-981-95-5099-9_13
17. Lyubashevsky, V., Seiler, G.: NTTRU: Truly fast NTRU using NTT. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2019**(3), 180–201 (2019). <https://doi.org/10.13154/tches.v2019.i3.180-201>, <https://tches.iacr.org/index.php/TCHES/article/view/8293>
18. Micciancio, D., Nicolosi, A.: Efficient bounded distance decoders for barnes-wall lattices. In: *2008 IEEE International Symposium on Information Theory*. pp. 2484–2488 (2008). <https://doi.org/10.1109/ISIT.2008.4595438>
19. Schnorr, C.P., Euchner, M.: Lattice basis reduction: improved practical algorithms and solving subset sum problems. *Math. Program.* **66**(2), 181–199 (Sep 1994). <https://doi.org/10.1007/BF01581144>, <https://doi.org/10.1007/BF01581144>
20. Schwabe, P., Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Seiler, G., Stehlé, D.: CRYSTALS-KYBER. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>
21. Simon, M.K.: *Probability Distributions Involving Gaussian Random Variables: A Handbook for Engineers, Scientists and Mathematicians*. Springer-Verlag, Berlin, Heidelberg (2006)
22. of Standards, N.I., Technology: Module-lattice-based key-encapsulation mechanism standard. (Department of Commerce, Washington, D.C.), Federal Information Processing Standards Publication (FIPS) NIST FIPS 203 (2024), <https://doi.org/10.6028/NIST.FIPS.203>
23. Zhang, J., Feng, D., Yan, D.: NEV: Faster and smaller NTRU encryption using vector decoding. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology – ASIACRYPT 2023, Part VII*. Lecture Notes in Computer Science, vol. 14444, pp. 157–189. Springer, Singapore, Singapore, Guangzhou, China (Dec 4–8, 2023). https://doi.org/10.1007/978-981-99-8739-9_6

A Properties of the Matrix $\mathbf{W}_n$

Lemma 5 Let $\mathbf{W}_n = \begin{pmatrix} 1 & 1 \\ 0 & \phi \end{pmatrix}^{\otimes(k-1)} \in \mathbb{C}^{\frac{n}{2} \times \frac{n}{2}}$ be the generator matrix of the Barnes-Wall lattice BW_n , where $n = 2^k$ for $k \geq 2$, and $\phi = 1 + i$. For any index $j \in \{0, 1, \dots, \frac{n}{2} - 1\}$, let $w_H(j)$ denote the Hamming weight of the binary representation of j . Then the following properties hold:

- (1) $\mathbf{W}_n$ is an upper triangular matrix.
- (2) The j -th diagonal entry of $\mathbf{W}_n$ is exactly $\phi^{w_H(j)}$.
- (3) Every entry in the j -th row of $\mathbf{W}_n$ is divisible by $\phi^{w_H(j)}$.

Proof. Denote the base matrix as $\mathbf{B} = \begin{pmatrix} 1 & 1 \\ 0 & \phi \end{pmatrix}$. By definition, $\mathbf{W}_n = \mathbf{B}^{\otimes(k-1)} \in \mathbb{C}^{\frac{n}{2} \times \frac{n}{2}}$. Any row index j and column index c in $\{0, 1, \dots, \frac{n}{2} - 1\}$ can be uniquely represented in binary form as $j = (j_{k-2} \dots j_1 j_0)_2$ and $c = (c_{k-2} \dots c_1 c_0)_2$. By the properties of the Kronecker product, the entry at the j -th row and c -th column of $\mathbf{W}_n$ is given by:

$$(\mathbf{W}_n)_{j,c} = \prod_{l=0}^{k-2} \mathbf{B}_{j_l, c_l}.$$

Property 1: Suppose $j > c$. Then there exists a most significant bit position l such that $j_l = 1$ and $c_l = 0$. For this specific position, the matrix entry $\mathbf{B}_{j_l, c_l} = \mathbf{B}_{1,0} = 0$. Consequently, the entire product evaluates to $(\mathbf{W}_n)_{j,c} = 0$ for any $j > c$. Thus, $\mathbf{W}_n$ is an upper triangular matrix.

Property 2: For the diagonal entries, we have $j = c$, which implies $j_l = c_l$ for all $l \in \{0, \dots, k-2\}$. The j -th diagonal entry is therefore $\prod_{l=0}^{k-2} \mathbf{B}_{j_l, j_l}$. Given $\mathbf{B}_{0,0} = 1$ and $\mathbf{B}_{1,1} = \phi$, each bit $j_l = 1$ contributes a factor of ϕ , while each bit $j_l = 0$ contributes a factor of 1. Since the number of bits where $j_l = 1$ is exactly $w_H(j)$, the diagonal entry is exactly $\phi^{w_H(j)}$.

Property 3: Consider an arbitrary entry $(\mathbf{W}_n)_{j,c}$ in the j -th row. If $(\mathbf{W}_n)_{j,c} = 0$, it is trivially divisible by $\phi^{w_H(j)}$. If $(\mathbf{W}_n)_{j,c} \neq 0$, it must hold that $\mathbf{B}_{j_l, c_l} \neq 0$ for all l . Specifically, for every bit position l where $j_l = 1$, we must have $c_l = 1$ (otherwise $\mathbf{B}_{1,0} = 0$). At these positions, $\mathbf{B}_{j_l, c_l} = \mathbf{B}_{1,1} = \phi$. This means for every 1 in the binary representation of j , the product $\prod_{l=0}^{k-2} \mathbf{B}_{j_l, c_l}$ accumulates a factor of ϕ . Since there are $w_H(j)$ such bits, the overall entry $(\mathbf{W}_n)_{j,c}$ is a multiple of $\phi^{w_H(j)}$. $\square$